

2 第二章 双雄对战：网络通信

第一章从整体视角勾勒了在线系统的层次结构。要把这样的整体构想逐步实现为成熟系统，通常先构建一个只保留核心交互和关键约束的最小可行系统（Minimum Viable Product, MVP），用简单请求验证通信路径、状态处理和异常处置是否成立，再逐步扩展功能与规模。

很多人都有过这样的体验：和朋友一起编辑同一份在线文档时，自己这边已经输入了一段话，对方的界面上却要过一会儿才出现；或者在联机游戏里，自己明明已经躲到掩体后面了，却还是被对方击中。

之所以会出现这种“对不上”，是因为在线服务不是在一台设备上跑一个程序，而是在每个用户的设备上都有一份独立运行的客户端，各自保存着一份当前的状态。用户的操作要先变成一条消息，经过网络传到对方那边，对方的状态才会更新。消息传递需要时间，在这段时间里，两边看到的状态自然就不一样。

状态暂时不一样还只是表面问题。更麻烦的是：如果两边都按自己看到的状态来判断结果，同一件事就可能得出两个不同的结论。比如文档编辑时同一句话被两个人同时改掉，交易时一边以为付了钱另一边以为没收到，游戏里一边觉得打中了另一边觉得躲过去了。结果不一致，是所有在线系统都必须解决的核心问题。

要避免这种分歧持续扩大，系统需要打通一次操作从发出到确认的完整链路：先把消息送到正确的程序，再协调各端状态的推进，最后形成双方都认可的结果。因此，本章依次回答三个问题。第一，两个分布在不同设备上的程序如何找到对方，并让接收方从传输的数据中恢复出一条完整消息。第二，各端状态暂时不一致时，怎样使它们继续围绕同一结果推进，而不是越差越远。第三，一次操作是否已经生效，应以谁的判断为准，又怎样让双方确认这一结果。它们分别对应建立连接、同步状态和确认结果，构成本章的主线。

本章以双人在线游戏为贯穿案例。游戏中的每个客户端都根据最近收到的状态版本绘制画面，网络传输所需的时间会使各端在同一时刻掌握不同版本。连续画面把原本隐藏在消息传输和状态更新中的变化直接呈现为角色位置或操作结果的差异，使通信、同步和裁决机制所处理的问题都能对应到具体的状态变化。如图 9 所示，服务器和客户端 B 已经进入状态 v42，客户端 A 仍停留在状态 v41；同一角色在 A 端画面中暴露在掩体外，在 B 端和服务器中却已隐蔽到掩体后方。

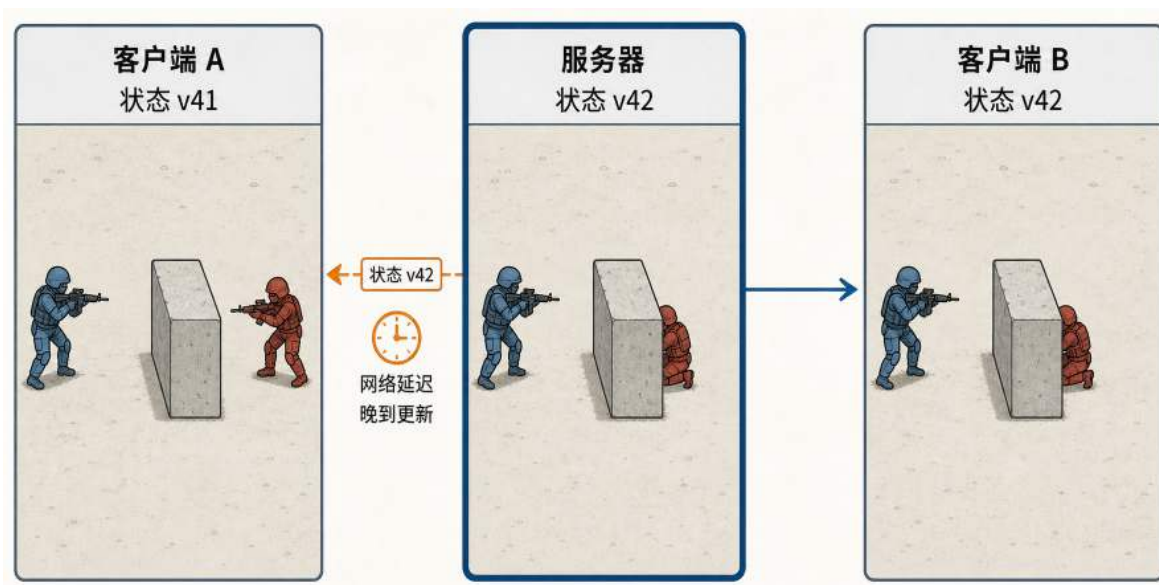


图 9: 网络延迟造成的状态版本差异: 客户端 A 仍显示 v41, 服务器与客户端 B 已进入 v42。

在图示时刻, 客户端 A 仍按照状态 v41 绘制画面, 因此它可能认为目标尚未躲到掩体后方, 并据此发出攻击请求。客户端 A 只能向服务器上报这次攻击, 不能根据本地画面直接认定攻击已经命中。

服务器收到攻击请求后, 以自己保存的状态 v42 和游戏规则为依据进行校验, 并记录最终结果。随后, 服务器把确认后的最新状态和攻击结果发送给两个客户端。客户端 A 收到这些信息后, 校正本地显示的角色位置和攻击结果, 各端由此按照服务器确认的状态继续交互。

协作文档、交易系统和即时通信也需要经过类似的处理: 先由系统统一确认操作结果, 再把确认后的状态同步给各参与方。因此, 游戏案例中的通信、状态同步和结果确认方法, 也可以用于处理其他共享状态系统中的类似问题。

本章以最小原型逐步演进: 先建立双端通信, 再由 P2P 暴露等待、信任和连接规模问题, 引出权威服务器; 随后处理超时、断连和重复请求, 并比较不同传输协议的取舍, 最后用输入、提交、同步、校正四个环节回收全章。

2.1 从单机循环到网络通信

理解多个程序如何通过网络通信协同, 需要先看清单台计算机内部, 一个程序如何接收输入、更新状态并呈现结果。在最简单的单机程序中, 这些操作由同一个进程依次完成, 不需要等待网络消息, 也没有其他独立程序参与。它的运行逻辑可以用一个循环概括:

单进程状态循环（概念示例）

```
1 for {
2     input := readInput()      // 读取本地输入
3     updateState(world, input) // 更新唯一一份本地状态
4     render(world)             // 把当前状态呈现出来
5 }
```

循环包含三个基本环节：`readInput` 读取用户输入，`updateState` 根据输入更新本地状态，`render` 把当前状态呈现到屏幕上。三者共同构成程序的主循环。

在单进程环境中，状态处理比较简单：程序只维护一份当前状态，所有逻辑都读写同一块内存；事件顺序由主循环自然确定，按到达的先后依次处理。由于状态只有一个副本，输入处理的结果也只在本地生效，不会影响其他独立运行的程序。

第二个独立程序实例加入后，两者的操作需要共同影响同一份共享状态。实例 A 的输入和画面运行在设备 A 上，实例 B 的输入和画面运行在设备 B 上；两个实例各自保存一份本地状态，无法再像单进程那样直接读写同一块内存。A 的操作要反映在 B 的画面中，必须先通过通信通道从 A 发送到 B；消息到达之前，B 只能按照旧版本继续呈现。由于网络传输需要时间，两个实例在同一时刻掌握的版本可能不同：A 可能依据较早的版本判断一次操作已经生效，B 却依据较新的版本认为状态已经改变。如果两端继续从各自保存的状态独立计算，同一操作就可能产生不同结果。要避免这种分歧继续扩大，两个实例必须交换操作或状态，并依据共同的信息更新各自保存的本地副本。

同一台机器上也可以启动两个彼此独立的程序进程，用来模拟分布在两台设备上的交互场景。两个进程分别维护自己的输入和状态，不能直接读取对方的数据，必须通过通信通道交换操作。每个进程仍然运行自己的主循环，但状态更新不再只依赖本地输入，还要纳入对端输入。双端的基本循环如下：

双进程交互循环（概念示例）

```
1 for {
2     local := readInput()      // 读取本地输入
3     send(conn, local)         // 把本地输入发给对端
4     remote := receive(conn)   // 等待对端的输入
5     merged := merge(local, remote) // 合并双方输入
6     updateState(world, merged) // 更新本地状态副本
7     render(world)            // 画面呈现
8 }
```

与前面的单进程循环相比，`readInput`、`updateState` 和 `render` 三个基本环节没有改变，双端循环新增了 `send`、`receive` 和 `merge` 三个步骤。`send` 把本地输入发送给对端，`receive` 获取对端输入，`merge` 将双方输入组织为当前一轮的共同输入，随后才能更新本地状态。

这些新增步骤使两个独立程序实例能够处理同一轮操作，也改变了主循环继续执行的条件。

`receive` 一直在等待对端输入而无法返回，本轮状态更新因而停滞；两端收到并处理输入的时刻不同，状态推进的进度可能暂时不一致；如果对端发送的数据超出预期范围，还可能把不可信输入带入后续计算。双端原型由此暴露出三个具体困难：等待消息会阻塞推进，两份状态可能产生分歧，对端提交的数据不能直接作为共同结果。

等待、状态分歧和输入可信性将在后续原型演进中逐步处理。在讨论这些问题之前，两个程序首先要能够找到对方，并从网络传来的字节中恢复出完整消息。下一节先建立这一通信前提；随后，点对点原型讨论两端如何同步推进，集中式服务器架构再解决由谁确认结果的问题。

2.2 网络通信基础：建立传输通道与消息边界

两个独立程序要交换第一条完整消息，需要依次完成三件事：确定对端程序的通信位置，建立一条可以读写字节的传输通道，再从连续字节中恢复出应用程序能够处理的消息。前两件事涉及 IP、端口、socket 和 TCP 连接，第三件事由序列化与消息分帧完成。完成这些工作，只说明消息已经可以在两个程序之间传递；状态如何同步、结果由谁确认，仍要在后续原型中解决。

2.2.1 建连：从地址端口到传输通道

一个程序不能直接读取另一台设备上进程的内存。要把数据送到对端程序，发送方先要确定目标主机和服务入口，再通过操作系统提供的网络接口建立连接。

IP 地址用于标识主机的网络接口。数据进入网络后，沿途路由器根据目标 IP 地址选择下一跳，最终把数据包送到目标主机。一台主机可以同时运行网页服务、聊天程序和游戏程序，仅有 IP 地址还不能确定数据应当交给哪个程序。

端口号用于区分主机上的通信端点。在常见的传输层协议中，端口号是一个 16 位编号，取值范围为 0-65535。服务器通常在约定端口等待连接，客户端的源端口则由操作系统临时分配。IP 地址和端口通常写成“IP: 端口”：IP 确定目标主机，端口进一步确定主机上的服务入口。

确定通信端点后，应用程序还需要一个使用网络能力的接口。操作系统为此提供了套接字（socket）抽象。程序通过 socket 创建通信端点、配置地址、收发数据和关闭连接，不必直接操作数据包或网卡。

应用程序使用的语言库会把这些操作转换为 socket 系统调用。操作系统同时返回一个标识 socket 的句柄；在类 Unix 系统中，这个句柄通常是文件描述符（file descriptor, fd）。内核中的 socket 是一个具体的通信对象，记录所用协议、本地与远端地址、连接状态、发送缓冲区和接收缓冲区等信息。因此，socket 是应用程序进入内核网络功能的入口，不是协议栈中的一个层次。

socket 可以使用不同的传输层协议。当前通信原型采用 TCP，由传输层处理丢失和乱序，使这一阶段可以集中验证建连和消息解析。这只是当前阶段的选择。有些数据必须按序到达，有些数据过期后便没有继续等待的价值；后一类场景可以考虑使用以独立数据报为交付单位的 UDP。第 2.6 节将完整比较两种协议的适用条件。

确定使用 TCP 后，一条连接还需要同时记录协议和两端地址。对本节讨论的普通 TCP 连接，内核用五元组加以区分：< 协议，源 IP，源端口，目的 IP，目的端口 >。IP 和端口标识

通信端点，五元组则把本地端点、远端端点和传输协议组合起来，标识一条具体连接。在连接两端，源地址与目的地址的角色互换，描述的仍是同一对通信端点。服务器同时接入多个客户端时，也依靠这些不同的连接记录把字节交给相应 socket；这一过程将在第 2.4.2 节展开。

端点和协议确定后，双方才能建立 TCP 连接。监听方先在本机地址和端口上创建监听 socket；发起方连接该地址；监听方接受连接后，得到一个专门用于本次通信的已连接 socket。监听 socket 负责接入新连接，已连接 socket 才负责后续读写。

在 Go 中，`net.Listen` 创建监听端点，`Accept` 和 `DialTimeout` 建立连接，并返回实现了 `net.Conn` 接口的连接对象。下面的概念代码把建连、原始字节往返和资源关闭放在同一个过程里。

两个程序实例的 TCP 建连与原始字节往返（概念示例）

```
1 // 主机 A: 监听并等待另一端接入
2 ln, err := net.Listen("tcp", ":8080")
3 if err != nil { /* handle */ }
4 defer ln.Close()
5
6 conn, err := ln.Accept()
7 if err != nil { /* handle */ }
8 defer conn.Close()
9
10 buf := make([]byte, 1024)
11 n, err := conn.Read(buf) // 读取当前可用的一段字节
12 if err != nil { /* handle */ }
13 if _, err := conn.Write(buf[:n]); err != nil { /* handle */ }
14
15 // 主机 B: 连接主机 A，并完成一次原始字节往返
16 conn, err := net.DialTimeout("tcp", "hostIP:8080", 3*time.Second)
17 if err != nil { /* handle */ }
18 defer conn.Close()
19
20 if _, err := conn.Write([]byte("probe")); err != nil { /* handle */ }
21 reply := make([]byte, 1024)
22 n, err := conn.Read(reply)
23 if err != nil { /* handle */ }
24 _ = reply[:n] // 交给后续处理逻辑
```

主机 A 在 8080 端口监听，`Accept` 为接入方返回已连接的 `net.Conn`；主机 B 使用主机 A 的地址和端口发起连接。连接建立后，两端通过 `Read` 和 `Write` 收发字节。示例把收到的内容原样返回，只用于验证通道是否可用，尚未解释这些字节对应什么业务操作。

应用程序调用 `Write` 后，字节经系统调用进入 socket 的发送缓冲区，再由 TCP、IP、链路

层和网卡送入网络。对端按相反方向处理数据，将字节放入接收缓冲区，接收程序再通过 Read 取出。TCP 的两个方向各自维护字节序列，因此两端可以同时发送和接收数据，这种通信方式称为全双工。两个方向仍会共享主机和网络资源。

Write 返回描述的是本次系统调用的结果，不能据此认定对端程序已经处理了数据。第 2.5.2 节将进一步区分本机写入、对端接收和业务生效。

一次字节传输需要经过应用程序、系统调用、socket 缓冲区、TCP/IP 协议栈和网卡。图 10 将两端的数据路径与应用层、内核之间的职责分工放在同一结构中。

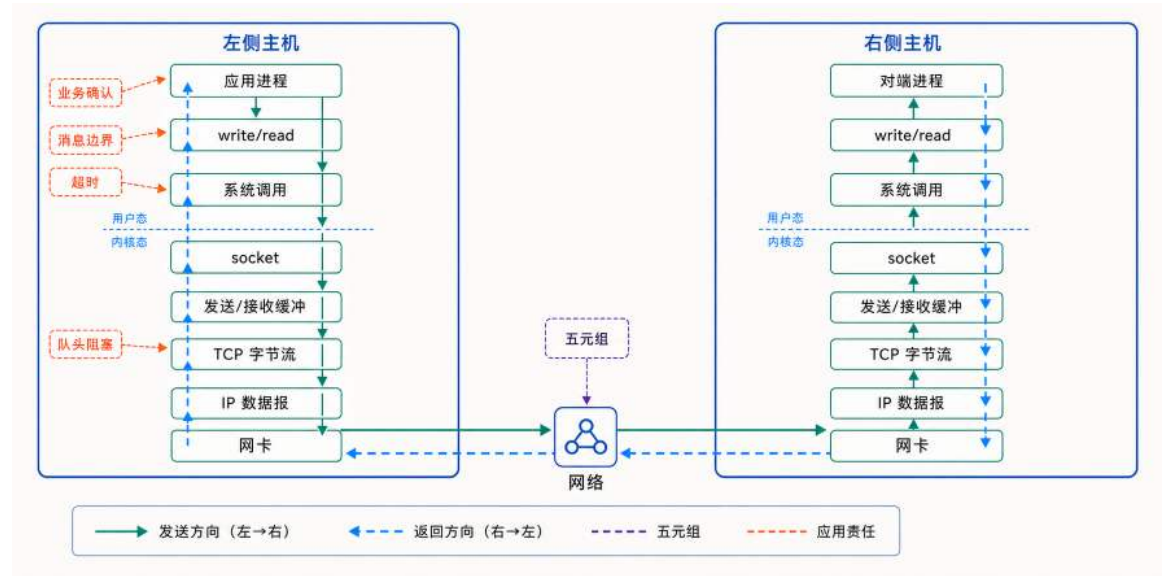


图 10: socket 与协议栈的职责分工：内核沿 TCP/IP 路径在两个通信端点之间传递字节，应用协议负责定义消息结构和业务完成条件。

TCP 在这里负责按序交付字节。下一小节先解决字节流中的消息边界；读取应等待多久、业务何时生效，分别在第 2.5.3 节与第 2.5.5 节展开。按序交付还会使后续数据等待缺失的前序数据，这一队头阻塞问题将在第 2.6 节讨论。

2.2.2 消息边界：如何在字节流中恢复完整消息

在线系统的业务逻辑通常以一条完整的请求或事件为处理单位，例如文档编辑操作、交易指令、即时消息和游戏输入。要把这些对象送入 TCP 通道，应用层协议需要完成两项工作：规定字段如何编码，并规定一条消息从哪里开始、在哪里结束。前者由序列化解解决，后者由分帧解决。

发送前，程序先把包含类型、标识和参数的结构化对象编码成字节序列；接收端取得完整字节后，再把它还原成对象。这两个过程分别称为**序列化**（serialization）和**反序列化**（deserialization）。JSON、Protocol Buffers 和自定义二进制格式都可以完成这种转换。

序列化只决定对象怎样表示，不能保留 TCP 中的消息边界。设发送端连续发送 M1 (attack) 和 M2 (move)。应用程序虽然分别调用两次 Write，TCP 传输的却是连续字节 attackmove。接

收端第一次 Read 可能取得 attackmo，第二次只取得 ve。如果程序把每次读取都当作一条消息，就会得到不存在的指令和无法解析的残片。

图 11 将无边界的读取结果与长度前缀分帧放在同一传输过程中，说明消息边界需要由应用层协议恢复。

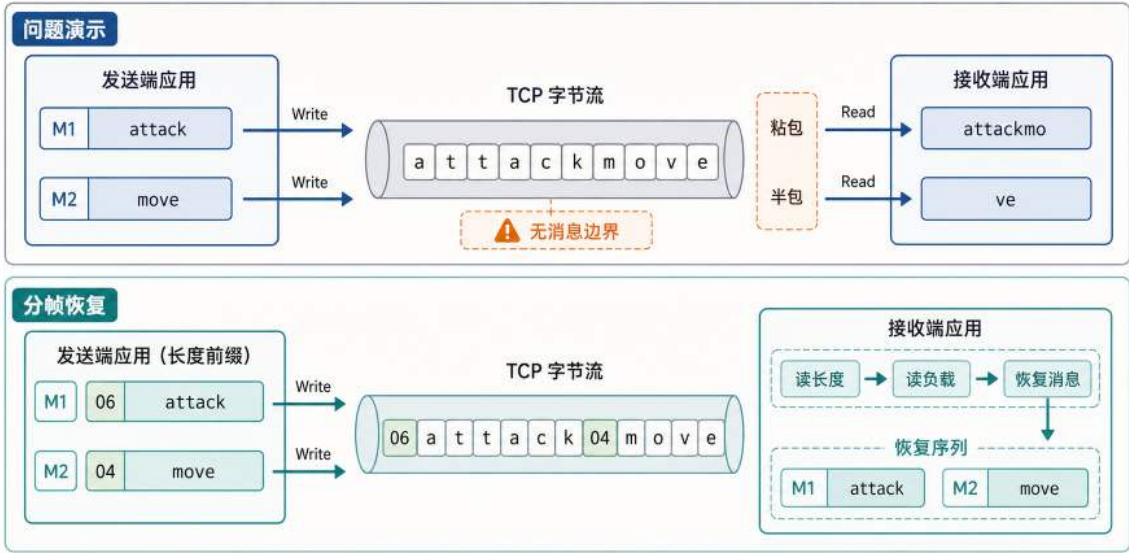


图 11: TCP 字节流与长度前缀分帧：发送端的写入次数与接收端的读取次数没有对应关系；长度前缀使接收端能够从连续字节中恢复消息。

多条消息的字节可能在一次读取中合并，通常称为**粘包**；一条消息也可能分成多次读取，通常称为**半包**。它们不是 TCP 故障，而是字节流不保留应用层分组的直接结果。Read 只返回接收缓冲区中当前可读的字节，读取长度与发送端调用 Write 的次数没有一一对应关系。应用层不能用一次 Read 的返回位置判断消息在哪里结束。

一种常用的分帧方法是在每条消息前放置固定宽度的长度字段，记录后续负载的字节数。教学示例可以把 M1 写成 06 attack、把 M2 写成 04 move；实际实现通常使用固定宽度的二进制整数。接收端先读长度，再持续读取指定数量的负载字节，取得完整负载后才交给业务逻辑。无论一次读取包含多条消息，还是一条消息需要多次读取，接收端都可以按长度字段恢复边界。

长度前缀的最小实现还要处理三个细节。长度字段需要使用双方约定的字节序；头部和负载都可能分多次到达，因此要读满指定长度；在申请负载缓冲区之前，还要检查长度上限，避免异常长度造成过大的内存分配。下一节的锁步输入同步也不能无限等待对端消息，因此示例为读取设置了截止时间。截止时间只用于限制等待，超时后的连接状态将在第 2.5.3 节详细讨论。

长度前缀分帧与限时读取（概念示例）

```
1  const maxFrameSize uint32 = 1 << 20 // 示例上限: 1 MiB
2
3  func writeFull(conn net.Conn, data []byte) error {
4      for len(data) > 0 {
5          n, err := conn.Write(data)
6          if err != nil { return err }
7          if n == 0 { return io.ErrShortWrite }
8          data = data[n:]
9      }
10     return nil
11 }
12
13 func sendFrame(conn net.Conn, payload []byte) error {
14     if uint64(len(payload)) > uint64(maxFrameSize) {
15         return ErrFrameTooLarge
16     }
17     var header [4]byte
18     binary.BigEndian.PutUint32(header[:], uint32(len(payload)))
19     if err := writeFull(conn, header[:]); err != nil { return err }
20     return writeFull(conn, payload)
21 }
22
23 func recvFrame(conn net.Conn, timeout time.Duration) ([]byte, error) {
24     _ = conn.SetReadDeadline(time.Now().Add(timeout))
25
26     var header [4]byte
27     read, err := io.ReadFull(conn, header[:])
28     if err != nil {
29         if read > 0 { return nil, ErrBrokenFrame }
30         return nil, err
31     }
32
33     n := binary.BigEndian.Uint32(header[:])
34     if n > maxFrameSize { return nil, ErrFrameTooLarge }
35
36     payload := make([]byte, int(n))
37     if _, err := io.ReadFull(conn, payload); err != nil {
38         return nil, ErrBrokenFrame
39     }
40     return payload, nil
41 }
```


`binary.BigEndian` 使两端按同一种顺序解释四字节长度字段。`io.ReadFull` 会持续读取，直到填满头部或负载；`writeFull` 则处理一次写入未完成全部字节的情况。`maxFrameSize` 是用于说明检查位置的示例上限，实际值应由应用协议允许的最大消息确定。同一连接上的完整帧还应由单一写流程串行发送，避免不同消息的头部和负载交错。

`sendFrame` 接收的 `payload` 是序列化后的字节，`recvFrame` 返回完整负载后，调用方再执行反序列化。长度字段只负责恢复边界。实际协议还可以在消息头中加入版本、消息类型和序列号，用于兼容协议升级、分派处理逻辑和标识消息所属轮次。

如果读取因超时返回，并且程序尚未取得任何头部字节，调用方可以重新设置截止时间后再读。一旦已经消费部分头部，或者头部完整但负载没有读完，解码位置便停在一条消息中间，不能把剩余字节当作下一条消息的头部。示例返回 `ErrBrokenFrame`，调用方应关闭并重建连接；具体的错误分类和重连处理将在第 2.5.3 节展开。

至此，两个程序已经能够建立 TCP 连接，并交换一条边界清楚的结构化消息。这并不保证两端状态一致，也没有回答发生分歧时由谁确认结果。下一节把通信通道接入状态更新过程，从最简单的 P2P 直连开始讨论这些问题。

2.3 P2P 直连原型：输入同步与状态一致

点对点直连 (Peer-to-Peer, P2P) 允许两个程序实例直接交换数据，不经过中间节点。少量参与者时只需维护少量连接，前文介绍的消息序列化和分帧机制均可直接复用。不过，消息能够到达，并不等于两端已经得到一致的运行状态；接收方还需要解释消息，并将其纳入本地状态更新。

一种做法是直接发送完整状态或状态增量，由接收方覆盖相应数据。另一种做法是只发送引起状态变化的输入，让两端分别计算新状态。本节采用后一种方法：网络负责同步输入，本地程序负责更新状态。由此，**输入同步是通信机制，状态一致是该机制期望得到的结果**。

2.3.1 锁步输入同步：机制与成立条件

如果两端不等齐同一帧的输入就各自推进，该帧使用的输入集合就会不同，状态更新随之分叉。为了避免这种分歧，可以让两端以完全相同的输入集合同步推进，这种方法称为锁步输入同步 (Lockstep)。

锁步把系统运行过程划分为连续的逻辑帧。逻辑帧采用固定的模拟步长，独立于各自的本地呈现频率。在第 $n+1$ 帧中，参与端 A 和参与端 B 分别产生输入 A_{n+1} 与 B_{n+1} ，并将带有帧号的输入发送给对方。两端收齐同一帧的输入后，再从相同的状态 S_n 出发执行同一个更新函数：

$$S_{n+1} = \text{Update}(S_n, A_{n+1}, B_{n+1}).$$

图 12 展示了一个逻辑帧内的数据流。跨越网络的是两条输入消息，状态更新则分别发生在两个程序实例内部。因此，锁步并不是把一份新状态从一端复制到另一端，而是让两端依据相同的输入独立计算下一帧状态。

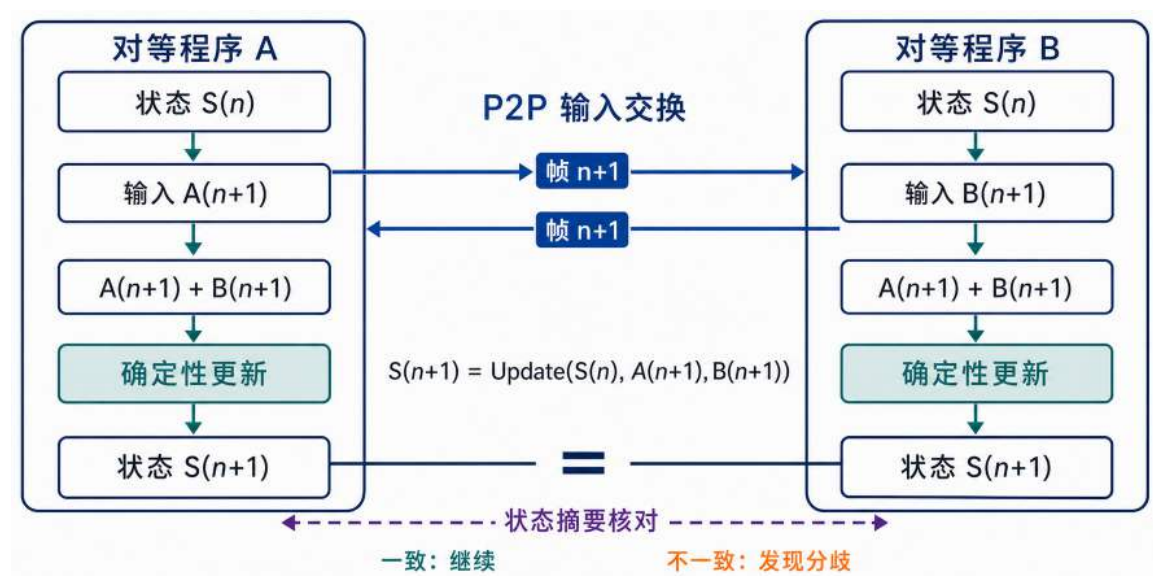


图 12: P2P 锁步输入同步：两个对等程序实例交换带帧号的输入，并在相同状态上执行同一更新函数，分别得到下一帧状态；状态摘要用于发现两份状态是否产生分歧。

上述过程能够维持状态一致，需要满足三个条件。第一，两端必须从相同的初始状态开始。第二，每条输入都必须携带逻辑帧号；某一帧的输入尚未收齐时，该帧不能进入状态更新。第三，更新过程必须具有确定性，即相同状态与相同输入在两端产生相同结果。随机数种子、运算顺序、时间步长以及可能产生平台差异的数值计算，都需要纳入确定性约束。

即使两端从相同状态出发，并使用相同输入和更新规则，程序缺陷、随机数使用不一致或数值计算差异仍可能使结果逐渐分叉。直接交换完整状态进行核对会产生较大的通信开销，因此，两端可以定期为当前状态计算一个较短的摘要，例如 checksum 或 hash，只交换摘要进行比较。摘要相同，表示本轮核对没有发现差异；摘要不同，表示两端状态已经分叉。摘要只能发现问题，不能判断哪一端正确，也不能修复状态。后续恢复仍需要重新加载双方认可的状态快照，或者终止当前会话。

将锁步机制写入程序时，每一帧需要完成四个动作：采样本地输入、发送本地输入、接收对端输入，再用双方输入更新状态。本地输入在一帧内只能采样一次；只要对端输入尚未到达，本帧就不能更新状态，也不能进入下一帧。

发送和接收的顺序同样重要。如果两端都先等待接收，而对方也尚未发送，两个程序就会相互等待。因此，两端都应先发送本地输入，再接收对端输入。下面的 `exchangeInput` 明确展开了这一顺序，其中 `encodeInput` 和 `decodeInput` 表示输入消息的序列化与反序列化，`validateFrame` 用于检查收到的输入是否属于当前帧。

锁步输入同步 (概念示例)

```
1 type InputMsg struct {
2     Frame int    // 输入所属的逻辑帧
3     Action Input // 本帧的具体操作
4 }
5
6 func exchangeInput(
7     conn net.Conn,
8     local InputMsg,
9     timeout time.Duration,
10 ) (InputMsg, error) {
11     // 先把本地输入序列化、分帧并发送
12     payload, err := encodeInput(local)
13     if err != nil {
14         return InputMsg{}, err
15     }
16     if err := sendFrame(conn, payload); err != nil {
17         return InputMsg{}, err
18     }
19
20     // 发送完成后, 再等待对端同一帧的输入
21     payload, err = recvFrame(conn, timeout)
22     if err != nil {
23         return InputMsg{}, err
24     }
25     return decodeInput(payload)
26 }
27
28 state := initialState()
29 frame := 0
30 for {
31     // 本帧只采样一次; 重试时仍发送同一份输入
32     local := InputMsg{
33         Frame: frame,
34         Action: sampleLocalInput(),
35     }
36
37     var remote InputMsg
38     for {
39         var err error
40         remote, err = exchangeInput(
41             conn, local, 50*time.Millisecond,
42             )
```

正常情况下, `InputMsg` 先被序列化并封装成长度前缀帧, 再由 `sendFrame` 发往对端。对端使用 `recvFrame` 取出完整消息并解码。两端都取得当前帧的本地输入和对端输入后, 才调用 `updateState` 更新状态。这条路径把本章前面介绍的 socket、字节流、序列化和分帧连接到了实际的状态更新。

如果交换过程超时, 本帧已经采样的 `local` 必须保持不变, 重试时仍然发送同一份输入, 不能为相同帧生成新的操作。如果超时发生在一条帧只读取了一部分之后, 当前连接已经无法正确识别下一条消息的起点, 需要先重建连接再重发本帧。只要当前帧的双方输入没有收齐, 状态和帧号都不能继续推进。

2.3.2 P2P 的局限: 等待、信任与连接规模

锁步输入同步降低了传输完整状态所需的带宽, 但也使系统推进依赖所有参与者。只要一端的输入延迟到达, 另一端就必须等待; 网络抖动或短暂离线会直接停住共同的逻辑帧。因此, 锁步系统的推进速度受最慢参与者限制。

第二个问题是结果的可信性。两个参与端都在本地计算状态, 其中任何一端都可能修改输入或更新逻辑。状态摘要可以证明两份状态不同, 却不能证明哪一份状态正确。对于竞争结果、账务变更等需要多方共同认可的关键状态, P2P 结构缺少一个独立于参与者的确认位置。

第三个问题是连接规模。双人 P2P 只需要一条连接; 若 N 个参与者两两直连, 则系统需要维护 $N(N-1)/2$ 条连接, 每个参与者需要与其余 $N-1$ 个参与者通信。参与者增加时, 连接管理、输入分发和等待关系都会迅速复杂化。

因此, P2P 适合参与者较少、彼此可信且网络环境相对可控的场景。随着参与者增加, 或者运行结果需要得到共同认可, 系统必须进一步解决两个问题: 如何降低参与者之间的连接复杂度, 以及如何建立独立于参与者的状态确认机制。

2.4 CS 架构与权威服务器设计

P2P 原型暴露的等待、信任和连接规模问题, 使系统需要引入独立服务器。下面先说明客户端与服务器的职责如何变化, 再按照连接管理、状态仲裁和状态分发的顺序展开服务器内部处理。

2.4.1 从 P2P 到权威服务器

CS (Client-Server) 架构通过引入独立服务器同时回应上述需求。客户端不再彼此建立连接, 而是将输入意图提交给服务器; 服务器检查输入是否合法, 依据自身维护的状态推进系统, 再将确认后的结果返回客户端。在这一模型中, 服务器承担**唯一真相源** (single source of truth, SSOT) 的角色, 影响共同结果的状态变更以服务器提交的版本为准。

图 13 从通信路径和状态确认两个维度呈现了这一转变。通信由参与者之间的多对多交换改为客户端与服务器之间的连接, 降低了参与者增加带来的连接复杂度; 状态则由各端分别计算和声明, 改为由服务器统一校验与提交, 使不同客户端能够依据同一份确认结果继续运行。

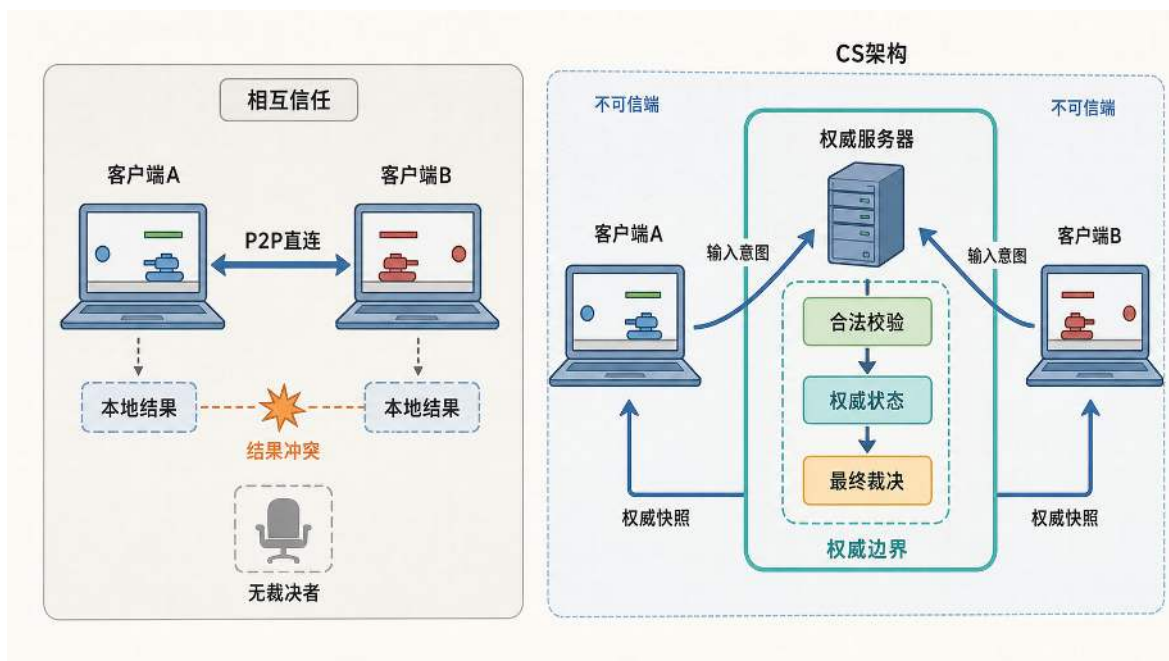


图 13: 从 P2P 到 CS 的通信路径与信任边界变化: P2P 让客户端直接交换并各自得出本地结果, CS 则把输入上报、规则校验和权威状态收拢到服务器。

这一架构变化重新界定了客户端提交信息的语义。在 CS 架构中, 客户端提交的是请求或操作意图, 服务器依据自身维护的状态完成合法性校验、状态变更和结果确认。电商系统接收下单请求, 协作系统接收编辑操作, 在线游戏接收移动或攻击指令。这些场景处理的业务对象虽然不同, 但均可采用相同的 CS 处理方式: 客户端提交操作请求, 服务器依据统一状态完成合法性校验, 并提交最终的状态变更。

在线游戏中的双人交互场景可以用来说明这一处理方式。首先明确该场景: 地图上只有两名玩家, 服务器维护二者的坐标、生命值、当前状态和对战会话。玩家可以移动; 两人靠近后, 客户端显示“可以发起对战”; 真正的请求仍要发到服务器, 由服务器重新检查距离、状态和会话占用, 再决定是否进入战斗。战斗本身先简化成单回合数值裁决: 服务器读取双方属性, 计算胜负, 更新生命值和战绩。

在这个场景中, 客户端负责采集输入和显示画面, 并上报移动、攻击等操作意图; 服务器依据权威坐标与规则判断移动能否执行、攻击是否命中, 并提交最终状态。由此, 不同客户端显示的局部状态可以暂时存在差异, 业务结果仍以服务器的计算为准。

实时游戏通常还会进一步优化显示效果, 用客户端预测 (client-side prediction) 减少本地操作的等待感, 用服务器和解 (server reconciliation) 把预测误差拉回权威状态, 用实体插值 (entity interpolation) 让远端角色的运动更平滑。¹ 这些技术用于改善客户端画面的连续性, 本节不深

¹Valve 对 Source 引擎多人网络模型中的 prediction、interpolation 与 lag compensation 有系统说明, 见 Valve Developer Community, Source Multiplayer Networking, https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking。关于网络游戏延迟补偿的经典教学文章, 也可参见 Glenn Fiedler, What Every Programmer Needs To Know About Game Networking, https://gafferongames.com/post/what_every_programmer_needs_to_know_about_game_networking/。

入讨论其具体算法，感兴趣的读者可以结合脚注文献进一步了解。当前讨论继续聚焦 CS 原型的基本职责：关键结果由服务器裁决，客户端维护用于显示和交互的本地副本。

服务器按照固定 tick 消费客户端输入并推进权威状态，客户端则发送输入并接收服务器返回的状态。下面的伪代码给出两端主循环的基本结构：

CS 架构伪代码结构

```
1 for { // 服务器端主循环 (tick 驱动)
2     // 从各连接收集截至当前 tick 的输入 (可能为空)
3     inputs := drainInputQueues()
4     // 推进一帧权威世界
5     updateGameState(inputs)
6     // 广播权威快照/增量
7     broadcastAuthoritativeState()
8 }
9
10 for { // 客户端主循环
11     // 采集本地输入并上报意图
12     localInput := getLocalInput()
13     sendInputToServer(localInput)
14     // 接收并渲染权威状态 (通常带超时/断线处理)
15     gameState, ok := receiveGameStateWithTimeout()
16     if ok { renderGameState(gameState) }
17 }
```

与锁步不同，CS 服务器不以收齐所有客户端的输入作为状态推进条件。网络读取过程将已经到达的输入写入队列，服务器在每个固定 tick 中处理当前可用的输入并更新权威状态，再按照独立的频率向客户端下发状态快照。因此，客户端输入采样、服务器状态更新和快照下发不必一一对应：一个 tick 可以处理多条输入，服务器也可以经过若干个 tick 再下发一次快照。这种解耦使系统能够根据交互延迟、服务器负载和网络带宽分别调整三个周期，同时由权威快照校正客户端暂时存在的状态差异。

前面的主循环说明了客户端与服务器如何交换输入和权威状态。要将这一循环实现为可运行的服务器，还需要把输入接收、规则校验、状态提交和结果返回落实到具体模块。为此，服务器内部划分为连接管理、状态仲裁和状态分发三项职责。

多个客户端的输入会以不同时间和顺序到达服务器，其中还可能夹杂重复、过期或不合法的操作。网络读取过程如果直接修改共享状态，连接抖动和异常输入就会进入状态更新路径，服务器也难以说明某次变更经过了哪些检查。因此，输入到达服务器后，需要先与权威状态的修改过程分开。

图 14 将这项分工放回一次持续运行的服务器处理过程中。连接管理先确认消息来自哪个连接和会话，再把解析后的操作意图写入事件队列。队列使网络接收可以继续运行，服务器主循环

则按照自己的 tick 消费当前可用事件；进入队列只表示服务器已经接收某项操作，并不表示该操作已经生效。

事件随后进入状态仲裁。状态仲裁依据当前权威世界检查移动、攻击等操作是否合法，并对同一 tick 内的事件进行排序和去重；只有通过这些处理的事件才能更新权威状态。这样，客户端提交的操作意图与服务器已经确认的业务结果被明确区分，多个客户端也会围绕同一份已提交状态继续交互。

状态分发读取已经提交的权威状态，根据可见范围和下发节奏生成快照，再发送给相关客户端。客户端用快照校正本地副本，并从校正后的状态产生下一轮输入。状态分发由此把一次服务器更新接回后续交互，使客户端暂时存在的画面差异不会演变为彼此独立的结果。

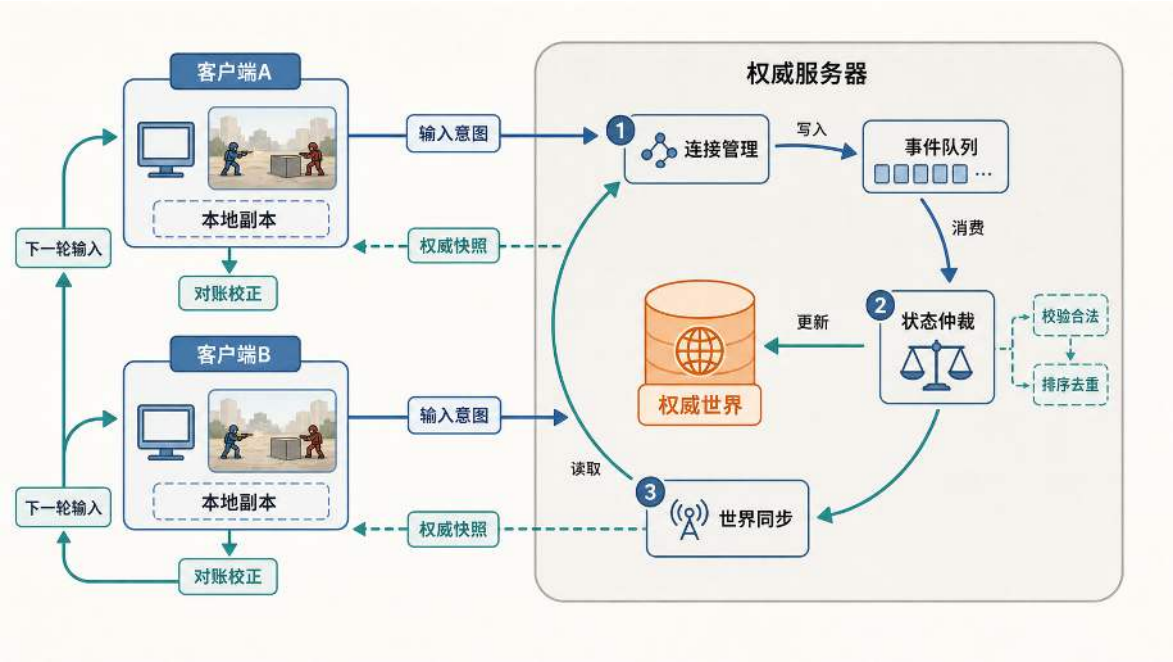


图 14: 权威服务器的持续处理路径：连接管理将输入写入事件队列，状态仲裁校验并提交状态变更，状态分发再用权威快照校正客户端副本并接入下一轮输入。

连接管理、状态仲裁和状态分发会在连续的 tick 中处理不同批次的输入，而非等待一项请求走完整条路径后再接收下一项请求。下文按照连接管理、状态仲裁和状态分发的顺序，分别说明每项职责面对的问题和实现要点。

2.4.2 连接管理：会话生命周期维护与读写调度

连接管理负责判断连接是否有效、连接归属哪个会话，以及网络读写应进入哪条有界队列。它不参与状态变更的合法性判定等业务规则校验；这些工作由状态仲裁完成。连接管理的核心任务是把网络输入转换为可处理的事件、把待发送状态交给独立写流程，并维护连接与会话从建立、运行到释放的完整生命周期。

接入循环与阻塞风险 一种常见错误是将连接接入与单个客户端的消息处理组织为顺序执行流程。服务器通过 `Accept` 建立一条连接后，立即进入该连接的消息读取与处理过程，只有处理过程结束，才会再次调用 `Accept`。如伪代码中展示的，如果该客户端长时间没有发送数据，读取操作将持续等待，服务器也无法接收后续连接；其他客户端的连接请求只能停留在接入队列中。

阻塞式连接处理的反例

```
1 for {
2     conn, err := listener.Accept()
3     if err != nil { continue }
4     handleClient(conn) // 若此处持续等待，后续连接将无法被接入
5 }
```

改进的关键是分离连接接入与消息读取。接入循环只负责建立连接和登记会话，完成后立即再次调用 `Accept`；已经建立的连接则进入独立的读取流程，读取流程将网络消息解析为输入事件并写入队列，服务器主循环在固定 `tick` 中消费这些事件。多个读取流程如何同时推进，将在下一章讨论。

内核连接与应用会话 客户端接入后，服务器需要同时维护两类状态。**内核态连接状态**随 `socket` 存在于内核，包括发送与接收缓冲区、TCP 序列号和拥塞窗口等；**应用会话状态**由服务器程序维护，包括玩家身份、最近一次收到消息的时间、应用层序列号以及重连窗口内需要保留的最小快照等。

两类状态的生命周期并不相同。网络中断会使原有 TCP 连接失效，玩家身份和对局进度却可能仍需保留，因此服务器不能只用 `socket` 表示一个客户端。连接管理需要为每个客户端建立一份可追踪的会话记录：会话 ID (Session ID) 标识这次应用会话，会话上下文 (Session Context) 保存会话运行期间需要持续维护的状态。

持有连接对象不能证明底层连接仍然有效，重新建立 TCP 连接也不会自动恢复应用会话。客户端重连后，服务器需要依据会话 ID 找到原有会话上下文，再把新连接与尚未结束的会话重新关联。

读写分离：网络等待不能阻塞世界推进 如果把 `socket` 读写直接放进世界主循环，一次网络等待就可能延迟整个共享状态的状态更新。最小服务器因此将网络读写与世界推进分开：连接读取流程持续从 `socket` 取出数据，把解析后的输入事件写入有界输入队列；世界主循环只在固定 `tick` 到来时消费队列并推进权威状态，再把待发送状态写入每个连接的有界发送队列；独立写流程负责真正写入 `socket`。输入队列或发送队列满时，系统必须明确选择等待或丢弃。对于可以被新版本覆盖的实时状态，系统可以淘汰过期的中间状态；不能丢弃的关键操作，则应让新的入队操作等待，或者返回明确的失败结果，使产生数据的一方随下游处理能力减速。这种把压力从下游队列传回上游的机制称为**反压** (backpressure)。若单个连接长期无法消费数据，服务器还应在超时后断开连接，避免它持续占用资源。本章先建立这项职责分工；当连接数量和消息速率上升后，如何同时推进多个读写流程、协调事件入队与消费并限制积压，将成为下一章讨论的主

要问题。

封装连接生命周期 工程上通常会把“连接句柄”和“会话上下文”打包成一个对象。这样服务器面对的就不再是零散的 `socket` 与全局变量，而是一份可追踪、可回收的会话状态。下面的 `clientConn` 把底层连接、会话与客户端标识、输入处理进度、连接响应时间以及收发队列放在一起；其中 `lastSeq` 表示会话上下文可以携带最近处理进度，真正决定某条输入是否生效的逻辑仍然留给状态仲裁模块。

连接管理的结构体定义

```
1 type clientConn struct {
2     id          int          // 唯一会话标识
3     clientID    int          // 会话关联的客户端
4     conn        net.Conn     // 当前底层连接
5     lastSeq     uint32     // 最近提交的输入序列号
6     lastReceivedAt time.Time // 最近收到合法消息的时间
7     lastPongAt  time.Time // 最近收到 Pong 的时间
8     lastSent    uint64     // 最近成功写入本机发送路径的状态版本
9     lastAacked  uint64     // 客户端已经确认处理的状态版本
10    eventQueue   EventQueue // 等待仲裁的有界输入队列
11    sendQueue    SnapshotQueue // 等待写入的有界状态队列
12 }
13 // 实例化客户端连接对象
14 var clientA clientConn
15 var clientB clientConn
```

完成这一层封装后，连接管理围绕 `clientConn` 执行两个步骤。`handleInput` 在收到合法的完整输入时更新最近接收时间，并将输入写入事件队列；`handleTick` 在每个服务器 `tick` 检查连接是否长时间没有合法消息，再由状态仲裁统一消费事件队列，随后推进权威状态并把待发送快照放入发送队列。对应的伪代码如下：

连接管理伪代码示例

```
1  const timeoutDuration = 30 * time.Second
2
3  // 收到并解析出一条完整输入后调用
4  func handleInput(c *clientConn, input InputEvent) {
5      c.lastReceivedAt = time.Now()
6      if !c.eventQueue.TryPush(input) {
7          applyInputBackpressure(c) // 不让输入无限积压
8      }
9  }
10
11 // 每个服务器 tick 调用一次
12 func handleTick(clients []*clientConn, frame uint64) {
13     now := time.Now()
14     active := make([]*clientConn, 0, len(clients))
15     for _, c := range clients {
16         if now.Sub(c.lastReceivedAt) > timeoutDuration {
17             handleDisconnect(c)
18             continue
19         }
20         active = append(active, c)
21     }
22     arbitrateFrame(active, frame) // eventQueue 只在仲裁阶段消费一次
23     advanceWorld(frame)           // 推进物理、计时器等服务器状态
24     queueGameStateForClients(active)
25 }
```

此时，连接管理模块已经能够维护每个客户端的会话状态，并把“读取是否超时”“连接是否健康”等问题显式化为可处理的事件。输入收集与超时检测分开后，服务器可以分别处理消息迟到、读取等待和连接超时，而不会让一次网络等待直接阻塞世界推进。

连接管理让双人原型从“一条裸 socket”推进到“可追踪的会话对象”：服务器不仅知道字节来自哪条连接，也知道这条连接属于哪个参与者、多久没有回应、输入应该进入哪个队列。但连接管理能维护的只是会话层面的状态；连接本身是否健康、写出去的字节是否真的被对端处理，这些更深层的可靠性问题，会在第 2.5 节集中面对。

2.4.3 状态仲裁：一致性防线与权威裁决

连接管理把网络读写整理成有序的输入事件后，状态仲裁依据业务规则校验这些事件，并决定相应的状态变更能否提交到服务器的权威状态。客户端上报的是意图，不是结果：位置变更上

报方向或坐标增量，而不是最终位置；关键操作上报请求，而不是计算结果。服务器只接受自己能验证的输入，并把通过校验的输入提交到权威状态。本章所说的权威状态，指的就是由服务器维护并作为所有客户端共同依据的那份状态。客户端可以持有显示用副本，但不能把副本上的推测结果直接变成事实。

状态仲裁还需要明确一条规则：同一 tick 内，服务器必须知道哪些输入参与本帧裁决。一个最小规则可以这样定：输入消息携带 `frame` 和 `seq`；服务器只接受当前裁决窗口内、序列号更新的输入；缺失输入按空操作或上一帧输入处理；迟到输入丢弃或转入下一帧，不能回头改已经广播的历史；同一 tick 内若有冲突，按固定玩家 ID 或服务器队列顺序处理。规则可以按业务调整，但必须在实现中固定下来，否则权威裁决就没有可执行的依据。

服务器将输入视为驱动权威状态机的命令，并根据 `tick` 与 `seq` 确定命令顺序。某个状态版本一旦提交并广播，后续到达的迟到消息就不能再改写这段历史。这些约束保证了三个基本不变量：一次会话只能创建一次，一次关键变更只能生效一次，已提交版本不能倒退。

以位置更新为例，服务器根据上一帧权威位置、速度上限与时间步长计算可达范围，并在必要时裁剪或拒绝超出范围的请求；以竞争结果裁决为例，服务器依据自身记录的权威状态与判定规则生成唯一结果。这样做的目的，是把决定公平性的计算收拢到服务器侧可记录、可复查的裁决路径中。

把状态仲裁放回下面代码中的状态更新逻辑，可以先分成“收集输入”和“提交裁决”两步：

状态仲裁伪代码示例

```
1 func collectForFrame(clients []*clientConn, frame uint64) map[int]InputEvent
2 {
3     inputs := map[int]InputEvent{}
4     for _, c := range clients {
5         input, ok := popLatestInput(c.eventQueue, frame)
6         if !ok || input.Seq <= c.lastSeq {
7             continue // 缺失或重复输入由服务器生成空操作
8         }
9         inputs[c.id] = input
10    }
11    return inputs
12
13 func arbitrateFrame(clients []*clientConn, frame uint64) {
14     inputs := collectForFrame(clients, frame)
15     for _, c := range sortedClientsByID(clients) { // 固定顺序
16         input, ok := inputs[c.id]
17         if !ok {
18             applyInput(&world, emptyInput(c.id, frame))
19             continue
20         }
21         if isValidInput(input, world.Player(c.playerID)) {
22             applyInput(&world, input)
23             c.lastSeq = input.Seq // 只在真实的新输入提交后推进
24         }
25     }
26 }
```

意图校验、序列号去重与权威提交都放在服务器一侧后，仲裁模块就能把“不可信输入”转成“可验证的状态变更”，信任边界也收紧到可控制的范围内。对双人原型而言，这一步意味着客户端不再提交“命中判定”“数值扣减”“胜负结果”这些计算结果，而是提交可校验的输入；真正改变共享状态的动作只发生在服务器的权威时间线上。

持久化：关键状态的保存与恢复 仲裁提交之后，状态还会遇到另一个落点问题：哪些只留在内存，哪些必须持久化。实时交互中的临时坐标、tick 内的临时变量可以只存在内存；而用户标识、配置、累计值等跨会话信息如果不落盘，一次服务重启就会使这些信息丢失，这在真实在线世界里不可接受。持久化同样服从信任边界：把存档交给客户端等同于放弃可审计性；关键资产应由服务器端保存。

存储层的职责是持久保存已经提交的关键状态，使服务重启或用户重连后能够恢复。为了识别同一业务请求的重复提交，请求通常携带稳定的唯一编号。请求编号与幂等处理的完整机制将在第 2.5.5 节展开。下面的存储接口只负责保存和加载服务器已经提交的结果，不判断请求是否重复。至于底层是 SQLite、PostgreSQL 还是云数据库，都不影响下面这层统一化接口。

持久化存储接口

```
1 type StoredEntity struct {
2     ID      string
3     Status  int
4     Data    []byte
5 }
6
7 type Store interface {
8     Load(id string) (*StoredEntity, error)
9     Save(e StoredEntity) error
10 }
```

当状态仲裁模块提交一次重要状态变更时，服务器调用 `Save` 写入持久化存储；客户端重连或重新登录时，再调用 `Load` 恢复必要字段。重复请求的识别以及是否复用已有结果，由状态仲裁层处理，不属于 `Store` 接口的职责。持久化的频率与方式本身是一种取舍：同步写入最直观，但会引入 I/O 延迟；更大规模系统通常会采用批量写回、异步写回与缓存层，第 3.3.3 节会继续讨论热数据、冷数据与缓存分层。

状态仲裁解决了输入“能否生效”的问题，持久化解决了关键结果“能否保留”的问题。但仅靠服务器内部提交和落盘还不够：裁决结果最终要回到客户端画面上，用户才能看到自己的操作产生了什么效果。

2.4.4 状态分发与副本收敛

状态仲裁把输入事件提交成权威状态之后，这些结果还需要到达客户端才能呈现在用户界面上。状态分发负责把服务器已经确认的结果以合适的范围、节奏和版本方式送回客户端。服务器算得再正确，如果客户端迟迟收不到，用户看到的仍然是旧版本。副本收敛要做的，就是让客户端本地保存的状态副本不断向服务器的权威状态靠拢。

状态分发首先要分清三种节奏。服务器的权威状态推进频率决定多久产生一份新的权威版本；状态发送频率决定多久向客户端推送一次状态更新；客户端的界面刷新频率决定用户多久看到一帧新的画面。这三个频率可以独立设置。例如，服务器以每秒 20 次的节奏推进权威状态、每秒发送 10 次状态更新，而客户端以每秒 60 帧的速度刷新界面；一次更新可以汇总多次状态变化，客户端则在相邻两次更新之间绘制多帧过渡画面。

在这个基础上，状态分发需要依次解决四个问题：发给谁、发什么、怎么衔接版本、客户端怎么显示。

发给谁，取决于客户端当前需要知道哪些实体的状态。服务器不会向所有客户端发送同一份全局状态，而是为每个客户端构造一份可见范围内的状态快照或增量。这样做有两个好处：一是减少不必要的网络传输，二是限制客户端获得与当前交互无关的信息。在在线游戏等空间化场景中，这种按可见范围筛选的机制常称为感兴趣区域（Area of Interest, AOI）过滤；在协作文档中，则可能体现为按光标位置或章节发送相关内容。

发什么，需要在全量快照和增量更新之间选择。全量快照包含一个实体或一片区域的完整状态，适合初次同步、重连恢复或纠错时使用；增量更新只发送自上一个已确认版本以来变化过的字段，适合稳定追踪时持续推送。为了让客户端知道增量应该接在哪个版本之后，每次更新通常会携带服务器 tick、版本号或序列号。客户端如果发现中间版本缺失，就不能盲目套用增量，而应请求全量快照或等待下一份可独立解释的完整状态。

版本衔接保证的是客户端收到的更新能够正确拼接到已有副本上。使用已确认版本作为增量的起点，会重复发送尚未确认的变化，但不会让客户端跨过自己尚未收到的版本。发送频率、队列容量和全量快照的成本，需要根据系统的实际负载和带宽条件来确定。

客户端侧的显示策略，则是在“及时反馈”和“最终一致”之间取得平衡。本地用户的操作可以立即在界面上呈现，以获得及时的操作感；但这种呈现只是预测，权威结果仍需等待服务器确认后才能确定。远端对象的状态更新到达后，客户端通常不会直接跳到新位置，而是在新旧两个权威版本之间做平滑过渡，以减少画面跳变。服务器的权威状态到达后，客户端需要用权威版本校正本地预测出的偏差，使本地副本最终与服务器保持一致。

服务器执行状态分发时，需要为每个客户端分别选择可见实体，并依据该客户端已经确认处理的状态版本生成快照或增量。状态分发主循环只负责生成更新并放入有界发送队列，真正的网络写入由每个连接的独立写流程完成。以下以游戏场景为例，给出状态分发的伪代码示意：

状态分发伪代码示例（游戏场景）

```
1 func queueGameStateForClients(clients []*clientConn) {
2     for _, c := range clients {
3         visible := queryVisibleEntities(world, c.playerID)
4         snapshot := buildDeltaForClient(c.playerID, visible, c.lastAked)
5         snapshot.Tick = world.CurrentTick
6         snapshot.Session = currentBattleSession()
7         snapshot.Events = collectPublicEvents(c.playerID)
8
9         if !c.sendQueue.TryPush(snapshot) {
10             // 队列已满：丢弃过期增量，改放一份可独立解释的全量快照
11             c.sendQueue.ReplaceWith(buildFullSnapshotForClient(c))
12         }
13     }
14 }
15
16 func runStateWriter(c *clientConn) {
17     for snapshot := range c.sendQueue.Items() {
18         _ = c.conn.SetWriteDeadline(time.Now().Add(writeTimeout))
19         if err := sendGameStateToClient(c.conn, snapshot); err != nil {
20             markConnectionAbnormal(c)
21             return
22         }
23         c.lastSent = snapshot.Version
24     }
25 }
26
27 func handleSnapshotAck(c *clientConn, version uint64) {
28     if version > c.lastAked && version <= c.lastSent {
29         c.lastAked = version
30     }
31 }
```

其中，`visible` 保存该客户端当前可见的实体，`buildDeltaForClient` 根据客户端已经确认的 `lastAked` 生成状态变化；随后写入服务器 tick、当前会话和公共事件，再把结果放入发送队列。

`runStateWriter` 为每个连接串行发送完整更新，并通过写截止时间限制慢连接占用写流程的时间；只有本地写入成功后才推进 `lastSent`。客户端成功应用状态后返回版本确认，服务器再由 `handleSnapshotAck` 推进 `lastAked`；确认版本不得超过服务器实际发送过的 `lastSent`，

避免客户端伪造未来版本。如果发送队列已满，示例用新的全量快照替换过期增量，使后续状态不依赖已经丢弃的中间版本。第 2.5 节将继续说明写入成功与应用层确认的区别。

实时交互场景的细化策略 在对延迟和画面连续性要求较高的实时交互场景中，上述基本策略还会进一步细化。感兴趣区域过滤需要精确到每个实体的可见范围，客户端预测需要区分本地操作和远端观测，实体插值需要设置合理的缓冲时长来吸收网络抖动。对操作时序敏感的场景还会引入延迟补偿 (lag compensation)：服务器保存最近一小段时间的权威历史状态，收到操作请求后在受限窗口内回看当时的状态，再判断操作是否在有效条件内发生。这样做可以改善用户的操作感受，但必须受到两项限制：回看所依据的状态必须来自服务器保存的权威历史，回看时长也不得超过预设上限，以防止客户端伪造历史状态或利用高延迟获得优势。

这些机制都依赖同一条时间线。为了减小不同机器之间的时钟偏差，真实系统可以采用网络时间协议 (Network Time Protocol, NTP) 进行时间同步，但权威裁决仍不能直接相信客户端本地时钟。更稳妥的做法，是以服务器 tick 和服务器接收窗口为准，客户端时间只作为辅助线索。状态分发与副本收敛真正要保证的，是所有关键事件最终落到服务器认可的逻辑时间线上；物理时钟对齐只能降低误差，不能替代服务器裁决。

状态分发与副本收敛补全了服务器到客户端的返回路径：服务器不只提交结果，还要把结果以合适的范围、版本和节奏送回客户端。客户端本地副本因此可以持续向权威状态靠拢，而不是停留在各自预测出的局部画面里。在实时游戏场景中，这组机制通常被称为“世界同步”，承担的也是同样的职责。

2.4.5 三模块协同：贯通服务器处理路径

至此，连接管理、状态仲裁和状态分发共同构成了服务器的完整处理流程：连接管理将网络输入转换为队列中的事件，状态仲裁将事件提交为权威状态，状态分发再将权威状态发送给客户端。它们不能消除延迟和作弊企图，但能让问题有明确归属：连接是否还存在，输入是否可生效，客户端副本是否已经追上权威状态。双人原型也由此从“能互相发送消息”推进到“能由服务器完成一轮权威处理，并让客户端依据确认后的状态继续交互”。

不过，上述职责划分只回答了“由谁接收输入、提交结果和同步状态”，暂时假设消息能够及时到达，并且发送方能够知道对方是否已经完成处理。真实网络并不提供这样的保证：一次写入成功可能只表示字节进入本机缓冲区，连接中断也未必能被立即发现，超时重试还可能让同一请求重复执行。因此，确定权威结果由谁提交之后，还要继续解决请求和结果怎样被业务双方可靠确认。

2.5 从可靠字节流到可确认的业务结果

本节继续追踪一条消息从应用程序写入到业务处理完成的路径，回答四个问题：连续的小量写入何时进入网络，发送操作成功能够证明字节到达了哪里，连接长时间没有响应时怎样发现异常，超时重试时怎样避免同一业务操作重复生效。这四个问题来自同一个根源：网络通信不能沿

用单机函数调用的假设。工程实践通常把“网络是可靠的”“延迟为零”“带宽无限”等错误假设概括为**分布式计算的八大谬误**（Fallacies of Distributed Computing）²。本节不展开完整列表，而是通过等待、断连、带宽开销和重复请求，检验其中与当前通信路径直接相关的假设。回答这些问题，需要重新检查前面用于 P2P 锁步同步和 CS 权威服务器的 TCP 通道。TCP 向应用程序提供**可靠、有序的字节流**，并通过**流量控制**避免接收端缓冲区被持续写满，通过**拥塞控制**在共享网络出现排队或丢包时调整发送速率。可靠性意味着丢失的字节会被重传，有序性意味着接收端只能按原始顺序读取字节，不能跳过中间缺口直接取得后续数据。

这些机制为当前 CS 原型提供了稳定的字节传输，长度前缀又使客户端和服务端能够从字节流中识别完整消息。到这里，系统仍然无法仅凭 TCP 判断连接是否继续可用、业务消息是否已经处理，以及重复发送是否会造成重复修改。在检查端到端的处理结果之前，还要先看消息是否已经离开发送端：应用程序完成一次写入后，操作系统仍要决定何时把缓冲区中的字节组成 TCP 数据段并发出。

2.5.1 小消息何时发出：Nagle 算法与时延取舍

应用程序调用 Write 后，写入的字节先进入操作系统的 TCP 发送缓冲区，再由 TCP 组成数据段并发送到网络。如果程序连续写入许多很短的消息，并且每次写入都立即产生一个 TCP 数据段，协议首部、确认报文和内核处理带来的开销可能远大于有效数据本身。发送端因而在“尽快发出当前数据”和“减少小数据段数量”之间作出取舍。

许多 TCP 实现提供 Nagle 算法来处理连续的小量写入。该算法由 John Nagle 在 RFC 896 中提出，其基本规则是：如果连接上没有已经发出但尚未收到对端 TCP 确认报文（ACK）的数据，新数据可以立即发送；如果发送缓冲区中仍保留着已发送但未确认的数据，后续的小量写入就先暂存在缓冲区中，直到发送端收到相应的 ACK，或者缓冲的数据已经足以组成一个较大的数据段。³

这里的 ACK 由对端 TCP 协议栈返回，只确认相应字节已经到达对端传输层，并不表示对端应用程序已经处理了这些字节。Nagle 算法可以减少连续小写入产生的数据段数量，但等待 TCP ACK 也可能增加后续小消息的发送时延。

位置更新、状态同步和交互指令等消息往往需要及时到达；当连接频繁产生这类小写入时，等待前一批数据确认可能使反馈变慢。这种等待不仅来自 Nagle 算法本身。TCP 的延迟确认机制（Delayed ACK）可以短暂等待后续数据或可捎带的反向数据再回复 ACK，实际等待时间由操作系统实现和连接状态决定，并不是固定的 40 ms⁴。Nagle 算法等待 ACK 才发新数据，延迟确认又可能等待更多数据才回复 ACK，两者叠加会放大小消息的等待时间。因此，对低时延优先的连接，

²对经典八项错误假设及其工程影响的列表，可参见 Microsoft Learn, *Smart Client—Building Distributed Apps with NHibernate and Rhino Service Bus*, 2010, <https://learn.microsoft.com/en-us/archive/msdn-magazine/2010/july/smart-client-building-distributed-apps-with-nhibernate-and-rhino-service-bus>。

³John Nagle, *Congestion Control in IP/TCP Internetworks*, RFC 896, 1984。该文提出了后来被称为 Nagle 算法的小数据段发送方法。

⁴TCP 对延迟确认的约束见 RFC 9293, *Transmission Control Protocol*, 第 3.8.6.3 节, <https://www.rfc-editor.org/rfc/rfc9293>。

通常选择关闭 Nagle 算法，使小量写入不必等待先前数据的确认。在 Go 中，`SetNoDelay(true)` 用于关闭这项等待；Go 创建的 `TCPConn` 默认已采用该设置，显式调用可以表明这里选择了低时延优先的发送策略：

降低小消息等待的 TCP 设置（概念示例）

```
1 if tcpConn, ok := conn.(*net.TCPConn); ok {  
2     _ = tcpConn.SetNoDelay(true)  
3 }
```

因此，是否关闭 Nagle 算法取决于通信目标：交互消息优先缩短等待时间，批量数据则优先减少数据段数量和处理开销。无论如何选择，它只改变发送端组织小数据段的方式，既不改变 TCP 的字节流语义，也不能证明消息已经到达并被对端处理。

2.5.2 写入成功能确认什么：从本机缓冲到业务生效

判断一条消息是否已经完成传递，需要区分本机写入、对端 TCP 接收和对端应用处理三个阶段。这三个阶段由不同组件完成，前一阶段成功不能证明后一阶段也已完成。

应用程序调用 `Write(data)` 时，首先把字节交给本机操作系统，并得到已写入字节数 `n` 和错误 `err`。只有当 `n == len(data)` 且 `err == nil` 时，这批数据才算完整写入本机 TCP 发送路径；如果 `n` 小于数据长度，则只有前 `n` 个字节被接收，程序还需要处理返回的错误和未写入的数据。即使完整写入成功，这些字节也可能仍在发送缓冲区中，尚未通过网络到达对端。

字节经过网络到达对端后，对端 TCP 会对已经接收的字节返回确认，即 TCP ACK。这个确认表示相应字节已到达对端 TCP 接收端，但不能说明应用程序已经读取这些字节。字节可能仍停留在接收缓冲区中，等待接收端程序调用 `Read` 读取；该程序随后还要解析消息、检查请求并执行相应操作，业务结果才会形成。

以“创建会话”请求为例，即使对端 TCP 已经确认收到请求字节，对端应用程序仍可能在读取或建立会话之前异常退出。此时传输层已经完成确认，业务层却没有产生有效结果。只有对端应用程序能够判断请求是否处理完成，这体现了**端到端论证**（end-to-end argument）⁵的逻辑：传输层可以保证其职责范围内的字节可靠、有序到达，但不能代替通信端点验证最终结果。

因此，写入返回成功只能表示数据已经完整交给本机 TCP（写入端成功）；它不能证明数据已到达对端，更不能证明业务处理已经完成，最终结果仍需由对端应用程序确认。

2.5.3 读取应等待多久：截止时间与超时处理

对 socket 的 `Read` 默认采用阻塞方式；如果暂时没有数据可读，调用它的执行流程就会继续等待。当网络读取与状态更新位于同一条顺序执行路径中时，这种等待还会延迟该路径上的后续处理。因此，应用程序需要为一次读取规定最长等待时间，使网络等待能够在确定的时刻返回给上层处理。

⁵端到端论证由 Saltzer、Reed 和 Clark 在 1984 年提出，见 *End-to-End Arguments in System Design*, *ACM Transactions on Computer Systems*, 2(4):277–288。

达到等待上限只说明程序没有在规定时间内读到数据，不能证明对端已经失效。相同的读取超时既可能由连接中断引起，也可能由排队、丢包重传或临时拥塞造成。应用程序应当把超时视为一种需要分类处理的运行状态，再结合消息类型、连续超时次数和最近一次有效响应决定后续动作，而不能在第一次超时后直接判定对端不可达。

Go 通过读取截止时间限制阻塞读取的持续时间。程序在调用 `Read` 前使用 `conn.SetReadDeadline` 设置绝对截止时刻；如果截止时刻到达时仍无数据可读，`Read` 返回超时错误，控制权重新交给应用程序。前文的 `recvFrame` 已经在读取消息头和负载前设置了截止时间。连接读取流程调用该函数后，还需要区分超时错误与其他读写错误，再把不同结果交给上层处理：

读取超时的识别与上报（概念示例）

```
1 func receiveOnce(conn net.Conn) error {
2     frame, err := recvFrame(conn, readTimeout)
3     if err == nil {
4         enqueueFrame(frame)
5         return nil
6     }
7     if errors.Is(err, ErrBrokenFrame) {
8         return ErrReconnectRequired
9     }
10    if errors.Is(err, ErrFrameTooLarge) {
11        return ErrProtocolViolation
12    }
13    if netErr, ok := err.(net.Error); ok && netErr.Timeout() {
14        return ErrReadTimeout
15    }
16    return err
17 }
```

`enqueueFrame` 只接收已经完整读取的消息。只有尚未消费任何帧字节的超时才作为 `ErrReadTimeout` 报告给上层，此时可以设置新的截止时间后继续等待；若 `recvFrame` 返回 `ErrBrokenFrame`，说明当前无状态解码函数已经丢失半条帧的读取进度，调用方必须进入重连流程，不能在原连接上重新从消息头开始读取；超过长度上限则属于 `ErrProtocolViolation`，服务器应拒绝消息并关闭连接。调用方再结合消息类型和连接健康状态决定继续等待、重试业务请求或重建连接。截止时间在触发后仍然有效，因此后续读取需要设置新的截止时刻，或者在允许长期等待时清除原有设置。

读取截止时间与 TCP 的重传超时（Retransmission Timeout, RTO）解决不同问题。RTO 位于传输层，用于决定发送端在未收到 TCP ACK 时何时重传数据段；读取截止时间位于应用层，用于限制当前读取操作最多等待多久。即使 TCP 仍在重传，应用层的读取也可能先达到截止时间。对实时状态更新，超时后可以暂时沿用上一份有效状态，等待下一次更新；对登录、结

算等关键操作，则可以进入有限次数的重试或重连流程。这样，系统既不会因一次读取无限等待，也不会把一次短暂延迟直接当作连接失效。

2.5.4 连接是否仍然可用：半开连接与健康检测

TCP 通常通过 FIN 或 RST 报文感知对端关闭连接。如果对端主机突然断电，或者中间网络中断使这些报文无法到达，本端在探测到故障之前仍可能保留原有连接状态。此时 socket 句柄仍然存在，Write 也可能先把数据写入本机发送缓冲区，但对端已经无法继续通信。这种两端连接状态不一致的情况称为**半开连接**。

半开连接会使服务端继续等待输入或发送状态，并保留对应的会话记录、发送缓冲区和资源配额。连接对象仍然存在或最近一次 Write 调用成功，都不能证明对端仍能接收并处理数据；系统还需要主动检查连接能否继续完成双向通信。

操作系统提供的 TCP keepalive 可以在连接空闲一段时间后发送探测报文，用于判断对端 TCP 是否仍可响应。它工作在传输层，探测时机由操作系统参数控制，也不能判断对端应用程序是否仍在正常处理请求。因此，仅依靠 TCP 的机制不能满足所有在线系统的连接健康判断要求。

当系统需要按照自身的交互周期判断对端状态时，可以让两端定期交换轻量级探测消息，并基于超时机制记录最近一次收到有效响应的时间，这就是**应用层心跳机制**。服务器定期发送 Ping，客户端收到后立即返回 Pong；服务器只有在收到 Pong 后才更新该连接的最后响应时间。如果超过设定时长仍未收到响应，就将连接标记为异常。以下代码给出主动检查的基本过程：

应用层心跳与连接健康检查（概念示例）

```
1  const (
2      heartbeatInterval = 5 * time.Second
3      heartbeatTimeout  = 15 * time.Second
4  )
5
6  func sendControlMessage(conn net.Conn, kind string) error {
7      return sendFrame(conn, []byte(kind))
8  }
9
10 // 建立连接后启动独立的周期检查流程
11 func runHealthCheck(c *clientConn) {
12     c.lastPongAt = time.Now()
13     ticker := time.NewTicker(heartbeatInterval)
14     defer ticker.Stop()
15
16     for now := range ticker.C {
17         if now.Sub(c.lastPongAt) > heartbeatTimeout {
18             markConnectionAbnormal(c)
19             return
20         }
21         if err := sendControlMessage(c.conn, "ping"); err != nil {
22             markConnectionAbnormal(c)
23             return
24         }
25     }
26 }
27
28 // 客户端收到 Ping 后返回 Pong
29 func handlePing(conn net.Conn) error {
30     return sendControlMessage(conn, "pong")
31 }
32
33 // 服务端收到 Pong 后更新最后响应时间
34 func handlePong(c *clientConn) {
35     now := time.Now()
36     c.lastPongAt = now
37     c.lastReceivedAt = now
38 }
```

`sendControlMessage` 沿用前文的消息分帧方法，在现有 TCP 连接上发送心跳控制消息。概念代码省略了入队动作；在实际连接管理中，心跳、状态快照和业务确认都应进入该连接的同一条写入流程，避免多个执行流程同时写入而破坏帧边界。`heartbeatInterval` 决定服务器主动发起检查的频率，`heartbeatTimeout` 规定允许连续收不到响应的最长时间。发送 Ping 失败，或者最后一次 Pong 已经超过该时间，都会使连接进入异常状态。这里的 `lastPongAt` 只记录主动探测的响应，`lastReceivedAt` 则记录任意合法入站消息；分开维护可以避免把“最近收到过业务消息”和“最近一次心跳探测已经成功”混为同一个判断。异常状态只表示系统在规定时间内无法确认连接健康；服务器可以先停止接受新的关键操作并保留重连时间，超过重连时间后再释放会话和相关资源。

因此，连接健康的判断依据不是“socket 是否存在”，而是“是否在规定时间内收到符合协议的有效响应”。超时机制提供了等待时间的上限，心跳和健康检测则在此基础上给出连接是否仍然可用的判断。

2.5.5 业务是否生效：应用层确认与幂等处理

前文区分了本机写入、对端 TCP 接收和对端应用处理三个阶段。这一区分带来一个直接问题：发送方如何判断请求已经产生了预期的业务结果。坐标、状态值等周期性信息可以由后续更新覆盖；会话创建、结算确认和关键状态变更等操作则必须得到明确结果，否则客户端无法决定是否继续后续流程。

业务处理完成后，由接收方返回包含请求编号和处理结果的确认消息，这种机制称为**应用层确认** (application-level ACK)。发送方发出请求后将其标记为待确认，只有收到编号匹配的确认消息，才把该请求标记为完成。TCP ACK 由接收端内核产生，应用层确认则由业务程序在处理请求后产生，二者分别证明字节到达和业务处理这两个不同阶段。

如果发送方在规定时间内没有收到应用层确认，它无法仅凭超时判断请求停在哪一步：请求可能尚未到达，可能仍在处理中，也可能已经处理完成但确认消息丢失。为了从前两种情况中恢复，发送方通常需要重发请求；然而在第三种情况下，重发会使已经完成的操作再次到达服务器。例如，结算请求已经执行，但确认消息在返回途中丢失；若服务器把重发请求当作新操作再次执行，就会产生重复处理。

为避免重复副作用，发送方应为每项关键操作分配唯一请求编号，并在重试时保持编号不变。接收方保存已经处理的请求编号及其结果；相同编号再次到达时，接收方直接返回原结果，不再执行状态变更。这种使同一请求执行一次或多次都产生相同业务效果的机制称为**幂等处理** (idempotency)。

以客户端请求创建会话为例，这套机制需要客户端和服务端遵循两项相互配合的规则：客户端超时后重发同一编号的请求，服务器则依据该编号判断请求是首次到达还是重复到达。以下伪代码给出客户端等待确认和服务端返回既有结果的基本过程。

端到端确认与重复请求处理（概念示例）

```
1 // 客户端：重试时保持请求编号不变
2 func createSession() error {
3     req := newCreateSessionRequest() // req.ID 在每次新请求中唯一
4
5     for retry := 0; retry < maxRetries; retry++ {
6         if err := send(req); err != nil {
7             continue
8         }
9         ack, err := waitAck(req.ID, timeout)
10        if err == nil {
11            if ack.Status == "accepted" {
12                return nil
13            }
14            return ErrRejected
15        }
16    }
17    return ErrUnconfirmed
18 }
19
20 // 服务器：重复请求返回首次处理时保存的结果
21 func handleCreateSession(req Request) {
22     if result, ok := findHandledResult(req.ID); ok {
23         sendAck(req.ID, result)
24         return
25     }
26
27     result := processCreateSession(req)
28     saveHandledResult(req.ID, result)
29     sendAck(req.ID, result)
30 }
```

在这条路径中，`send` 成功仍然只表示字节已交给本机内核。客户端收到与请求编号匹配的 `accepted` 结果后，才能确认服务器已经建立会话；收到拒绝结果时则停止重试，并把业务失败返回给上层。`findHandledResult` 使服务器能够识别重复请求，`saveHandledResult` 则保存首次处理的结果，保证重发请求获得相同响应。伪代码省略了业务状态变更与请求结果记录之间的原子提交（两项操作要么同时生效，要么都不生效），以及服务器故障后的记录恢复；这些状态管理问题将在后续章节继续讨论。

在可能丢失消息并触发重试的网络中，仅靠发送和确认不能保证业务操作恰好执行一次。常

见做法是允许请求至少到达一次，再通过稳定的请求编号、结果记录和幂等处理，使重复到达不会重复产生业务效果。

2.5.6 通信逻辑如何复用：连接通道封装

分帧、读写超时和心跳检测并不只服务于某一种业务消息。如果会话请求、状态同步和结算确认的业务代码分别实现这些逻辑，不同处理路径可能采用不同的消息格式、等待时间和连接状态判断，增加维护和排查故障的难度。因此，可以在 TCP 字节流与业务处理之间增加连接通道，集中完成消息编解码、分帧收发、超时控制和响应时间记录。

连接通道只处理消息如何传递，不判断消息产生了什么业务结果。请求编号和确认消息可以通过连接通道传输，但等待哪一项确认、重复请求是否再次执行，仍由业务层决定。下面的概念代码用统一消息结构承载消息类型、请求编号和业务负载，并复用前文定义的 `sendFrame` 与 `recvFrame` 完成收发。

连接通道封装（概念示例）

```
1  type Message struct {
2      Type      byte
3      RequestID uint64
4      Payload   []byte
5  }
6
7  type ReliableConn struct {
8      net.Conn
9      lastPong time.Time
10 }
11
12 func (c *ReliableConn) Send(msg Message) error {
13     data, err := json.Marshal(msg)
14     if err != nil { return err }
15     return sendFrame(c.Conn, data)
16 }
17
18 func (c *ReliableConn) Recv(timeout time.Duration) (*Message, error) {
19     data, err := recvFrame(c.Conn, timeout)
20     if err != nil { return nil, err }
21
22     var msg Message
23     if err := json.Unmarshal(data, &msg); err != nil {
24         return nil, err
25     }
26     return &msg, nil
27 }
28
29 func (c *ReliableConn) RecordPong() {
30     c.lastPong = time.Now()
31 }
32
33 func (c *ReliableConn) Healthy(interval time.Duration) bool {
34     return time.Since(c.lastPong) < interval
35 }
```

`Send` 先把完整消息序列化，再交给 `sendFrame` 添加长度前缀；`Recv` 通过 `recvFrame` 限时读取完整帧，再将字节还原为消息结构。收到 `Pong` 消息后，连接管理逻辑调用 `RecordPong` 更

新时间；Healthy 只判断最近一次有效响应是否仍在允许时间内，不能保证连接之后一定可用。该封装只展示连接通道的职责，并不自动保证并发调用安全：每条连接仍应由单一写入流程调用 Send；若多个执行流程共享心跳状态，还需要使用下一章介绍的同步机制保护 lastPong。

RequestID 由连接通道原样传递，其业务含义仍由上层解释：客户端用它匹配确认消息，服务器用它查找已经处理的请求结果。这样，连接通道负责“怎样传递一条完整消息”，业务层负责“消息是否已经处理并生效”。

由此可以区分三层职责：TCP 在连接有效期间向应用程序提供有序字节流；连接通道恢复消息边界、限制等待时间并记录连接响应；业务层通过请求编号、应用层确认和幂等处理判断关键操作是否已经生效。这种分层没有改变 TCP 的按序交付语义。当较早的字节丢失时，后续字节仍需等待缺口恢复；下一小节将分析这种等待对不同消息的影响。

2.5.7 前序数据丢失会怎样：队头阻塞

TCP 按照字节在数据流中的顺序向应用程序交付数据。当后续内容依赖前序内容时，这种语义可以避免应用程序读到顺序颠倒的数据；相应的代价是，只要前面存在尚未恢复的字节缺口，后面已经到达的字节也不能提前交给应用程序。这段等待是否可以接受，取决于数据过期后是否仍有价值。会话创建、购买和结算等关键操作不能因为等待时间较长而跳过前序结果；坐标和状态值等高频信息则会持续更新，新值到达后，旧值通常已经失去使用价值。

设服务器依次发送状态更新 A、B、C，分别表示第 1、2、3 个版本的状态。应用程序把三条更新写入同一条 TCP 字节流后，TCP 实际传输的是承载这些更新的连续字节，而不是三份彼此独立的消息。如果承载 A 的部分字节在传输中丢失，接收端 TCP 可以暂存随后到达的 B、C 对应字节，但不能越过序号缺口把它们交给应用程序。发送端重传缺失的数据段并填补缺口后，接收端才能继续交付连续字节，连接通道随后才能解析出 A、B、C。这种由前序数据缺失而阻塞后续数据交付的现象称为**队头阻塞 (Head-of-Line Blocking, HOL)**。

在缺口恢复之前，客户端无法从这条连接取得 B、C 中的新权威状态，只能继续使用上一份已交付状态，或者显示本地预测和插值结果。对于必须完整处理的关键操作，等待重传是维持正确性的必要代价；对于能够被新状态覆盖的高频更新，等待过期数据会扩大一次丢失造成的延迟。要让后续更新不受前序缺口阻塞，需要采用彼此独立的交付单位。下一节将对比 TCP 的按序字节流与 UDP 的独立数据报，并给出两种交付方式的结构示意。

2.6 从按序等待到独立交付：TCP 与 UDP 的取舍

沿用上一节 A、B、C 的状态更新场景，TCP 必须先恢复承载 A 的字节缺口，应用程序才能继续取得 B、C。如果应用更关心最新状态，而不需要等待已经过期的旧状态，就需要让每次更新成为可以独立交付的单位。UDP 提供的正是这种**数据报**语义：应用程序每调用一次发送操作，就产生一份独立数据报；接收端也以数据报为单位取得数据，不需要先把它们拼成一条连续字节流。

在 UDP 路径中，A、B、C 分别放入三份数据报。若 A 数据报丢失，已经到达的 B 和 C 仍

可独立交给应用程序，客户端因而可以直接采用较新的 C 更新状态。这种及时性并非没有代价：UDP 不会自动补发 A，也不保证 B、C 按发送顺序到达。接收端需要依据序列号识别新旧状态，丢弃晚到的旧数据；确实不能丢失的消息则需要由应用层另行确认和重传。

两种交付方式最终改变的是应用程序取得新状态的时机：TCP 必须先恢复连续字节，UDP 则可以先处理已经到达的后续数据报。应用层再依据消息是否允许丢失，决定用新状态覆盖旧状态，或者为关键事件增加确认与重传；图 15 概括了这条从传输语义到应用决策的关系。

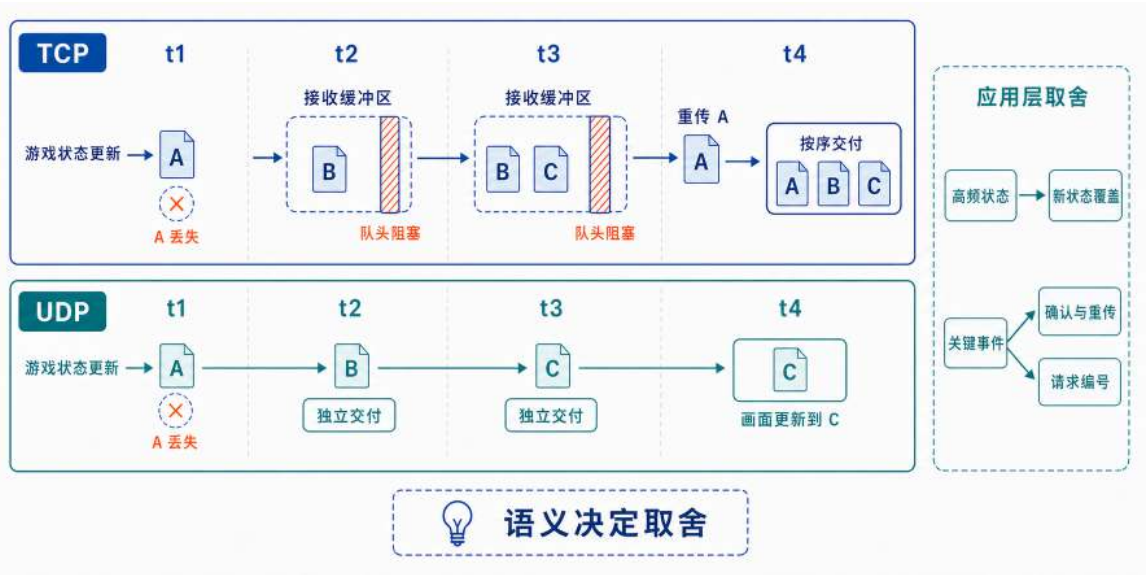


图 15: 前序数据缺失后的两种交付方式：A、B、C 表示应用层状态更新；TCP 填补承载 A 的字节缺口后才能继续按序交付，UDP 则可将已经到达的 B、C 数据报独立交给应用程序。

TCP 与 UDP 的差别并不是简单的理解为“可靠”与“不可靠”。TCP 在传输层统一实现丢失重传和按序交付，应用程序得到的是连续字节流；UDP 保留数据报之间的独立性，把是否重传、如何排序以及哪些消息可以丢失的决定更多交给应用层。选择传输方式前，应先判断一条消息过期后是否仍有价值，以及它的丢失是否会破坏业务规则。

坐标、状态值等高频信息通常符合“越新越有用”的特点。这类消息可以使用 UDP 或基于 UDP 的协议，并携带序列号；接收端保留较新的状态，忽略已经过期的旧状态。会话创建、关键操作和结算确认等关键事件则必须得到明确结果。它们可以通过 TCP 传输，也可以在 UDP 之上增加应用层确认、有限重传、请求编号和幂等处理；关键不在于协议名称，而在于最终交付语义是否满足业务要求。

独立交付也要求应用程序承担更多控制责任。第一，UDP 数据报可能丢失、重复或乱序，接收端必须使用序列号识别重复和过期消息。第二，过大的数据报可能触发 IP 分片，其中任一分片丢失都会使整份数据报无法交付，因此高频状态更新应控制单个数据报的大小。第三，UDP 不自带拥塞控制；发送方仍需限制发送速率，并在排队和丢包增加时主动减速。因此，UDP 避免了单条字节流中的队头阻塞，却没有消除丢包、拥塞和应用层可靠性问题。

选读：浏览器接入与多流传输 基于浏览器的应用通常使用 WebSocket 建立持续双向通道。WebSocket 在一条 TCP 连接上提供消息化的双向通信，WSS 再使用 TLS 保护传输过程。⁶由于底层仍是 TCP，前序字节缺失造成的等待不会因此消失；TLS 也只能保护传输内容，不能替服务器判断客户端上报的操作是否合法。

HTTP/2 在一条 TCP 连接上复用多条应用流，但底层字节缺口仍可能影响这些流；QUIC 则以 UDP 为承载，在其上实现加密连接、可靠传输、拥塞控制和多条独立流，使一个流的缺失数据通常不阻塞其他流。⁷这些协议改变了可靠性与顺序性的控制粒度，但仍不能替应用层决定哪些业务消息可以丢失、哪些必须确认。

本章的最小双人系统继续使用一条 TCP 通道承载全部消息，并在其上加入分帧、超时、心跳和端到端确认。UDP 在这里提供后续优化的判断标准：高频状态更重视及时性时，可以迁移到独立数据报；关键事件无论采用哪种传输方式，都必须保留可确认、可重试和可去重的端到端语义。

2.7 最小在线交互的完整流程

前几节已经依次建立通信通道、P2P 试验原型和 CS 权威服务器，并补充了超时、心跳、确认与幂等处理。这些机制不是零散的知识点，而是围绕同一条处理路径组织起来的。从用户发起操作到双方看到一致的结果，整个过程可以归纳为四个环节：输入、提交、同步和校正。

输入环节，用户的操作意图被封装为结构化消息，经过分帧、编号后通过网络进入服务器的处理队列。提交环节，服务器在自身维护的权威状态上校验操作的合法性，按既定顺序执行状态变更，并将结果写入持久化存储。只有提交完成的结果才算真正生效。同步环节，服务器将已提交的状态以快照或增量的方式，按可见范围和发送节奏分发给各个客户端。校正环节，客户端用收到的权威状态更新本地副本，修正此前预测产生的偏差，使两端最终收敛到一致的状态。

这四个环节共同构成了所有带客户端副本的共享状态系统的基本闭环。无论是协作文档、交易系统还是在线游戏，核心流程都可以对应到这四个环节上，区别只在于业务规则和状态对象不同。

双人在线游戏中的一次完整交互，可以把前面讲过的机制串联成一条可追踪的完整路径。流程从两名参与者相遇开始，到发起交互、服务器裁决、结果写回和会话释放结束，依次覆盖输入、提交、同步、校正四个环节。

2.7.1 相遇与会话建立：从可见交互到权威确认

第 2.4.4 节已经说明，状态分发需要根据每个客户端的可见范围选择同步对象。双人场景可以采用最简单的距离阈值：当两名参与者之间的距离小于设定值时，服务器将对方加入各自的可

⁶WebSocket 的握手、消息分帧和双向通信机制见 RFC 6455: The WebSocket Protocol, <https://www.rfc-editor.org/rfc/rfc6455>; TLS 1.3 的安全目标与握手机制见 RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3, <https://www.rfc-editor.org/rfc/rfc8446>。

⁷HTTP/2 的多路复用机制见 RFC 9113: HTTP/2, <https://www.rfc-editor.org/rfc/rfc9113>; QUIC 的流与多路复用机制见 RFC 9000: QUIC: A UDP-Based Multiplexed and Secure Transport, <https://www.rfc-editor.org/rfc/rfc9000>。

见列表，并在下一次状态同步中发送“新增可见实体”。这是 AOI 过滤在双人场景中的最小应用。

客户端收到新增实体的状态后，才在本地副本中创建对应的远端实体。网络通道负责传递状态，可见范围过滤负责选择同步对象，可见关系则以服务器保存的权威状态为准。因此，网络延迟或客户端本地数据变化只会暂时影响界面呈现，不会改变服务器认定的可见关系。

进入彼此视野只说明双方可以获取对方的状态，还不足以启动一次交互。两名参与者继续靠近并满足发起条件后，服务器可将双方标记为“允许发起交互”，再把这一状态通知客户端。客户端据此显示操作入口，但该提示只表示当前具备发起条件，不能作为交互已经建立的依据。

客户端发起交互时，只发送请求意图。服务器收到请求后，必须依据当前权威状态重新检查双方距离、参与者状态和会话占用情况；检查通过后，服务器才能创建交互会话（Session），并将双方从“自由活动”切换为“交互锁定”。这种再次检查是必要的，因为客户端显示操作入口之后，距离和会话状态都可能已经发生变化。

资格检查、会话槽占用、会话创建和参与者状态切换应当构成一次不可分割的状态迁移。否则，两个并发请求可能同时占用同一参与者。进入交互的通知不属于这次原子状态迁移：服务器应先提交会话状态，再向客户端发送通知；通知丢失时可以重发或查询，但尚未提交的会话不得提前显示为已经建立。第 3.2.1 节将进一步讨论这类需要把检查与修改合并处理的并发问题。

为处理响应丢失造成的重试，请求还需要携带唯一的 `request id`。服务器对相同标识返回同一会话结果，而不重复创建会话；客户端在收到已提交状态后再进入正式交互界面，超时则重试请求或查询会话状态。由此，“交互是否已经建立”取决于服务器提交的会话状态，而不取决于某一次通知是否成功到达。这正是输入环节中“消息可靠送达”和提交环节中“操作只生效一次”两条原则的具体体现。

2.7.2 结果裁决与结算：从规则校验到一致性写入

交互会话建立后，服务器依据会话中的权威状态处理双方输入并计算结果。参与裁决的数据包括各方当前状态、属性值和已经生效的临时效果。若规则包含随机因素，伪随机数也应由服务器生成并记录其种子或结果，使同一会话的裁决能够复查；客户端不得提供或修改决定结果的随机值。

对于带有时间条件的操作，服务器还要确定输入属于哪个裁决窗口。操作请求可以携带输入序号和客户端显示时间，但客户端时间只能作为辅助信息。服务器应以接收时间、逻辑帧和受限的历史窗口为依据：过旧请求按迟到规则处理，窗口内请求则可以使用服务器保存的历史状态进行延迟补偿。这样既能补偿正常网络延迟，又能防止客户端用自行声明的时间改写已经提交的历史。

裁决完成后，服务器更新状态值、累计值和会话状态，并生成待同步的结果事件。客户端可以预测操作反馈，却不能提交“命中”“生效”或“回避”等结果。将规则校验和结果提交集中在服务器，才能使共同状态具有唯一、可记录和可复查的裁决依据。这是提交环节的核心职责。

结算写入与会话释放 结算阶段必须先明确业务结果的提交位置。在双人单服原型中，服务器可以在一次事件处理中完成状态值、累计值和会话状态的迁移；需要在服务重启后保留的结果，

还应先写入可恢复的持久化记录。只有这些状态成功提交后，服务器才生成带有会话标识和结算版本的下行消息。客户端依据该消息展示结算结果，不再自行修改状态字段。

客户端返回 ACK，只能说明某个结算版本已经送达并被处理，不能决定结算是否成立。若 ACK 或下行消息丢失，服务器可以重发同一版本；客户端断线重连后，也应查询服务器最近提交的会话结果，而不是沿用断线前的本地画面。业务事实由服务器侧可恢复的提交记录确定，消息确认只用于判断是否需要继续发送。

服务器确认结算状态已经提交后，才能释放交互会话，将双方从”交互锁定”切回”自由活动”，并重新纳入常规的状态分发。随后的快照把新状态发送给双方及其可见范围内的其他参与者；客户端收到快照后清理已确认输入，并将本地副本校正到结算版本。若结算需要同时修改多个服务或多个存储分片，单个服务器内的状态迁移便不足以保证一致性，需要采用第 4.4.4 节讨论的分布式一致性机制。

2.7.3 闭环全景与跨场景迁移

走完整条流程可以看到，一次交互从发起到完成，始终围绕输入、提交、同步、校正四个环节展开。客户端产生输入事件，服务器对事件进行接收、排序和校验，再提交新的状态版本；服务器通过状态分发把已提交结果送回客户端，客户端再用权威版本校正本地副本的偏差。这四个环节分别回答了事件如何进入系统、结果在哪里生效、状态如何传播以及副本出现偏差后如何恢复。

同样的闭环也存在于其他共享状态系统中。以协作文档为例，用户的编辑操作是输入，服务端的合并与版本提交是提交，文档内容向所有协作者推送是同步，本地草稿与服务端版本的对齐则是校正。以交易系统为例，下单请求是输入，服务端的库存校验与扣款是提交，订单状态通知是同步，客户端页面与服务端订单状态的对齐是校正。业务对象和规则不同，但处理闭环的结构是一致的。

流程全景图 完整流程需要区分本地显示与权威提交。客户端发出操作请求后可以先显示预测反馈，但结果只有在服务器完成资格校验、会话锁定、结果裁决和状态写回后才正式生效。状态分发随后发送已提交的状态版本，客户端通过对账校正本地副本。图 16 按照最终 CS 原型展示输入意图、服务器提交和客户端收敛的先后关系；前文的 P2P 仅用于暴露双端裁决的问题，因此不进入该流程。

图中的关键区分是”消息送达”与”业务提交”。队列、重传和 ACK 可以提高消息送达的可靠性，却不能代替服务器提交业务结果；客户端预测可以缩短操作反馈时间，却必须允许后续快照纠正；持久化记录则使断线客户端能够恢复已经提交的状态。由此，输入顺序由服务器 tick 和裁决窗口确定，业务结果由服务器提交，客户端副本由状态分发和对账机制持续校正。

双人流程只包含少量连接和共享状态。扩展到多人同空间后，连接管理需要处理更多并发连接和 I/O 等待，状态仲裁需要处理同一 tick 内更密集的输入与状态访问，状态分发则需要承担更高的事件速率和可见范围计算成本。这些变化构成了第 3 章讨论并发问题的起点。

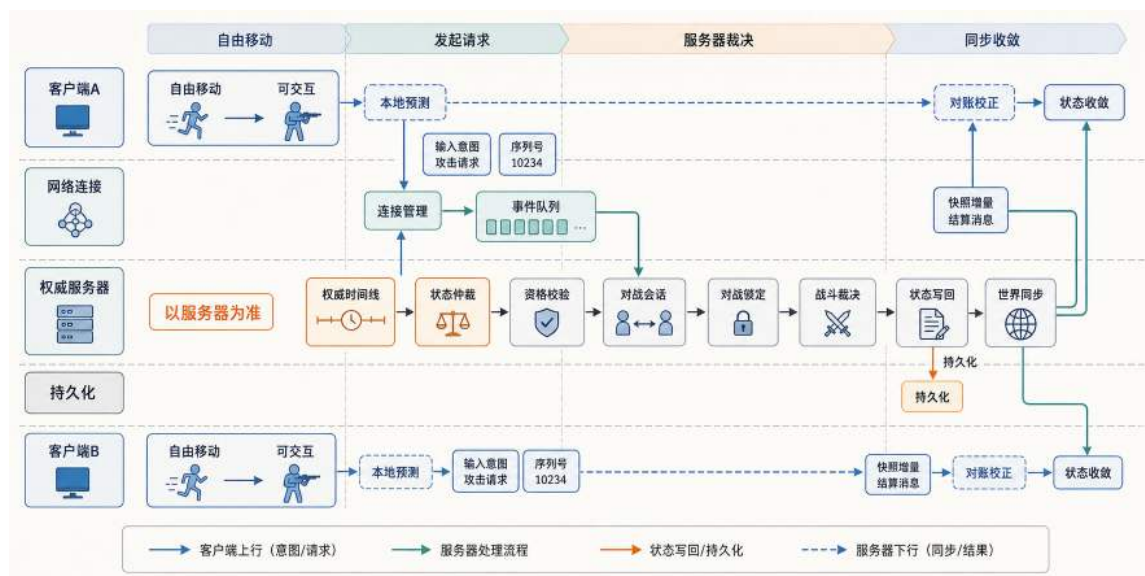


图 16: 双人交互案例的提交与收敛路径：客户端预测只改变本地副本；服务器完成裁决和状态写回后，结果才正式生效；世界同步与客户端对账使两端状态收敛到同一权威版本。

2.8 小结：网络通信与共享状态的最小闭环

本章围绕跨设备共享状态系统的三个基本问题展开：独立程序之间如何传递结构化消息，多份状态副本如何保持一致，以及操作结果如何被双方可靠确认。双人在线游戏作为贯穿案例，为这些问题提供了具体的观察场景，但讨论的机制和判断标准同样适用于协作文档、交易系统和即时通信等其他在线服务。

网络通信基础解决的是第一个问题。IP 地址和端口确定了通信端点，socket 提供了使用网络的编程接口，TCP 提供了可靠有序的字节流传输，序列化和长度前缀分帧则使应用程序能够从字节流中恢复出完整的结构化消息。在此之上，两个独立程序实例才有可能交换操作意图和状态信息。

P2P 直连和 CS 架构回答的是第二个问题。P2P 锁步输入同步展示了两端如何通过相同的输入和确定性更新获得一致的状态，同时也暴露了三个根本局限：推进速度受最慢端限制、缺少独立的状态确认位置、连接数随参与者平方增长。CS 架构通过引入权威服务器同时回应了这些约束：服务器作为唯一真相源统一校验输入和提交结果，客户端只维护用于显示和交互的本地副本。服务器内部进一步分解为连接管理、状态仲裁和状态分发三项职责，分别处理输入接收、结果提交和状态同步，客户端再依据权威版本完成校正，形成输入、提交、同步和校正四个环节。

TCP 可靠性与端到端语义解决的是第三个问题。TCP 的可靠是字节流层面的可靠，写入成功不代表消息已经到达对端应用，更不代表业务操作已经生效。半开连接、心跳检测、读取超时、应用层 ACK、request id 和幂等处理，共同把传输层的字节可靠交付提升为业务层的操作可确认、可重试、只生效一次。TCP 与 UDP 的取舍则取决于消息语义：关键操作和状态变更必须最终生效，高频状态更新则更重视及时性。

双人在线游戏的案例把这些机制串成了一条可以追踪的完整路径。沿着输入、提交、同步、

校正四个环节走一遍，读者可以看到通信机制、状态同步和端到端确认如何在一个具体场景中协同工作。下一章会把同一套闭环放到更多连接和更高消息速率下，讨论并发压力如何改变服务器的处理方式。

2.9 思考题

1. 为什么“TCP 可靠”仍然可能一次读到两条指令（或半条指令）？应用层应该补哪一层机制？
2. 锁步输入同步为什么会被最慢的一端拖慢？它适合哪些场景，不适合哪些场景？
3. 在权威服务器模式下，客户端仍然需要做哪些本地计算来降低感知延迟？哪些计算必须由服务器执行才能保证结果一致？
4. 为什么仅靠“本地检测到 close”不足以判断一条关键消息是否真的到达并生效？应该如何补齐端到端语义？
5. 在一个在线协作文档系统中，哪些状态必须持久化，哪些状态可以只存在内存？请说明理由。
6. 以电商下单为例，购物车更新、提交订单和支付确认三类消息分别更重视及时性还是最终生效？如果引入 UDP，应怎样分配传输与应用层可靠性？
7. 两名用户同时编辑同一段文档，用户 A 删除一个句子，用户 B 在同一位置插入一个段落，且 B 的操作消息晚到 120ms。请沿“分帧—连接管理—状态仲裁—状态分发—客户端校正”的路径说明服务器和客户端分别会看到什么、处理什么。
8. 在线游戏中的“世界同步”与本章讨论的“状态分发与副本收敛”是什么关系？请举出另一个领域中的同类机制。